

ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



HỌC THỐNG KÊ

Đồ Án Cuối Kỳ: Phát Hiện Văn Bản Do AI Sinh Ra
Sử Dụng DAIGT-V2 và DeBERTa-v3-base

Sinh viên thực hiện:

Phạm Ngọc Duy (23120035)
Huỳnh Đặng Ngọc Hân (23120042)
Lê Minh Nhật (23120067)

Giảng viên hướng dẫn:

TS. Ngô Minh Nhật
Lê Long Quốc

TP. Hồ Chí Minh - Ngày 29 tháng 05 năm 2026

Mục lục

Phân Chia Và Tiến Độ Hoàn Thành Công Việc	3
1 Giới thiệu đồ án	4
1.1 Quy Ước Nhãn	4
1.2 Mục Tiêu	4
1.3 Kiến Trúc Tổng Thể Pipeline	5
2 Cấu Trúc Thư Mục Và Vai Trò Từng Phần	6
2.1 Vai Trò Của Các Notebook Chính	7
2.2 Thứ Tự Chạy Đúng	7
3 Mô tả dữ liệu	7
3.1 Nguồn Dữ Liệu	7
3.2 Phân Phối Nhãn	8
3.3 Chia Dữ Liệu	9
3.4 Duplicate Và Leakage Check	10
4 Tiền xử lý Và Phân Tích Dữ Liệu	10
4.1 Kiểm Tra Chất Lượng Dữ Liệu	10
4.2 Feature Engineering Cho EDA	11
4.3 Các Hình EDA Chính	11
4.4 Nhận Xét Từ EDA	15
4.5 Các Bước Tiền Xử Lý	15
4.6 Tokenization	16
4.7 Phân Tích Token Length	16
4.8 Dataset Và DataLoader	16
4.9 Validate Pipeline	17
5 Mô Hình Baseline	17
5.1 TF-IDF + Logistic Regression	17
5.2 TF-IDF + SGDClassifier	18
5.3 TF-IDF + LightGBM	18
6 Mô Hình Chính: DeBERTa-v3-base	19
6.1 Tổng Quan DeBERTa	19
6.1.1 a) Disentangled Attention (Chú Ý Phân Ly)	19
6.1.2 b) Enhanced Mask Decoder (EMD)	19
6.1.3 c) DeBERTa-v3: RTGM + SiFT	19
6.2 Cấu Hình Mô Hình	19
6.3 Pipeline Mô Hình	20
6.4 Mean Pooling	20
6.5 Classifier Head	20
7 Fine-tuning Và Lựa Chọn Hyperparameter	21
7.1 Cấu Hình Cuối Cùng (Final)	22
7.2 Optimizer Và Layerwise Learning Rate Decay (LLRD)	22
7.3 Scheduler Và Warmup	23

7.4	Training Loop	23
7.5	Training Curve	23
7.6	Nhận Xét Training Curve	25
8	Đánh Giá DeBERTa Trên Held-out Test Set và so sánh mô hình	26
8.1	Nhận Xét	26
8.2	Validation Results	26
8.3	Test Results	26
8.4	Hình Biểu Đồ So Sánh	27
8.5	Nhận Xét Quan Trọng	30
9	Weighted Ensemble	30
9.1	Công Thức	30
9.2	Tối Ưu Trọng Số	30
9.3	Trọng Số Hiện Tại	31
9.4	Nhận Xét Trọng Số Ensemble	31
10	Error Analysis	32
10.1	Tổng Quan	32
10.2	Phân Tích False Positive (Human $\rightarrow$ AI)	32
10.3	Phân Tích False Negative (AI $\rightarrow$ Human)	33
10.4	Confusion Matrix	34
11	Web UI Demo	34
11.1	Tổng Quan	34
11.2	Luồng Xử Lý	34
11.3	Tính Năng Giao Diện	35
12	Chiến Lược Cải Thiện	36
12.1	High Impact	36
12.2	Medium Impact	37
12.3	Experimental	37
13	Kết Luận	37
13.1	Hoàn Thành Các Yêu Cầu	37
13.2	Kết Quả Đáng Chú Ý	38
13.3	Nhận Xét Tổng Thể	38
A	Phụ Lục A: Bảng Thuật Ngữ	38
B	Phụ Lục B: Danh Sách Hình Ảnh	40
C	Tài Liệu Tham Khảo	41
C.1	Dataset Và Bài Toán AI-generated Text Detection	41
C.2	Transformer, DeBERTa Và Fine-tuning	41
C.3	PyTorch Training Pipeline	42
C.4	Baseline TF-IDF, Logistic Regression, SGD Và LightGBM	42
C.5	Metrics Và Đánh Giá Mô Hình	43
C.6	Web UI Và Triển Khai Demo	43
C.7	Ghi Chú Về Tính Cập Nhật	43

Phân Chia Và Tiến Độ Hoàn Thành Công Việc

Bảng dưới đây mô tả phần việc chính, sản phẩm bàn giao và mức độ hoàn thành của từng thành viên trong nhóm. Việc phân chia được thiết kế theo pipeline end-to-end của đề án, từ phân tích dữ liệu, tiền xử lý, huấn luyện mô hình đến đánh giá và xây dựng demo.

Thành viên	Phần việc chính	Sản phẩm/đầu ra	Tiến độ
Phạm Ngọc Duy			
MSSV: 23120035	Phụ trách và tích hợp báo cáo/Web UI, phân tích dữ liệu ban đầu, kiểm tra chất lượng dữ liệu, phân phối nhãn và xây dựng các đặc trưng thống kê phục vụ EDA.	<code>web_ui/app.py</code> và nội dung báo cáo, <code>task1_eda.ipynb</code> , các hình EDA trong <code>outputs/eda/</code> , bảng thống kê đặc trưng văn bản.	100%
Huỳnh Đặng Ngọc Hân			
MSSV: 23120042	Phụ trách phân tích dữ liệu ban đầu, kiểm tra chất lượng dữ liệu, phân phối nhãn và xây dựng các đặc trưng thống kê phục vụ EDA, phụ trách tiền xử lý văn bản, chia tập dữ liệu, tạo fold stratified, tokenization.	<code>task2_pipeline.ipynb</code> , <code>train_clean.pkl</code> , checkpoint DeBERTa và log huấn luyện.	100%
Lê Minh Nhật			
MSSV: 23120067	Phụ trách baseline TF-IDF, Logistic Regression, SGDClassifier, LightGBM, ensemble, đánh giá mô hình, phân tích lỗi, fine-tuning model Deberta.	<code>task4_ensemble.ipynb</code> , các biểu đồ ROC/PR/confusion matrix, bảng so sánh metrics và nội dung báo cáo, <code>ablation-deberta.ipynb</code> , checkpoint DeBERTa và log huấn luyện.	100%

Bảng 1: Phân chia công việc và tiến độ hoàn thành của từng thành viên

Tất cả các thành viên đều hoàn thành phần việc được giao. Các kết quả trung gian được liên kết theo đúng thứ tự pipeline: EDA → tiền xử lý và chia dữ liệu → fine-tuning/huấn luyện baseline → ensemble, đánh giá và demo.

1. Giới thiệu đề án

Đề án xây dựng một hệ thống phân loại văn bản nhị phân nhằm xác định một đoạn văn bản là **do con người viết** hay **do mô hình AI sinh ra**. Bài toán được xây dựng trên bộ dữ liệu **DAIGT-V2** (lấy ý tưởng từ cuộc thi [Kaggle: LLM — Detect AI Generated Text](#)) và được triển khai theo pipeline end-to-end gồm:

1. Phân tích dữ liệu (EDA)
2. Tiền xử lý văn bản
3. Chia tập dữ liệu (train/validation/test)
4. Huấn luyện baseline truyền thống (TF-IDF)
5. Fine-tune Transformer (DeBERTa-v3-base)
6. Đánh giá mô hình và phân tích lỗi
7. Xây dựng giao diện Web UI demo bằng Gradio

1.1. Quy Ước Nhãn

Trong toàn bộ báo cáo, quy ước nhãn được sử dụng như sau:

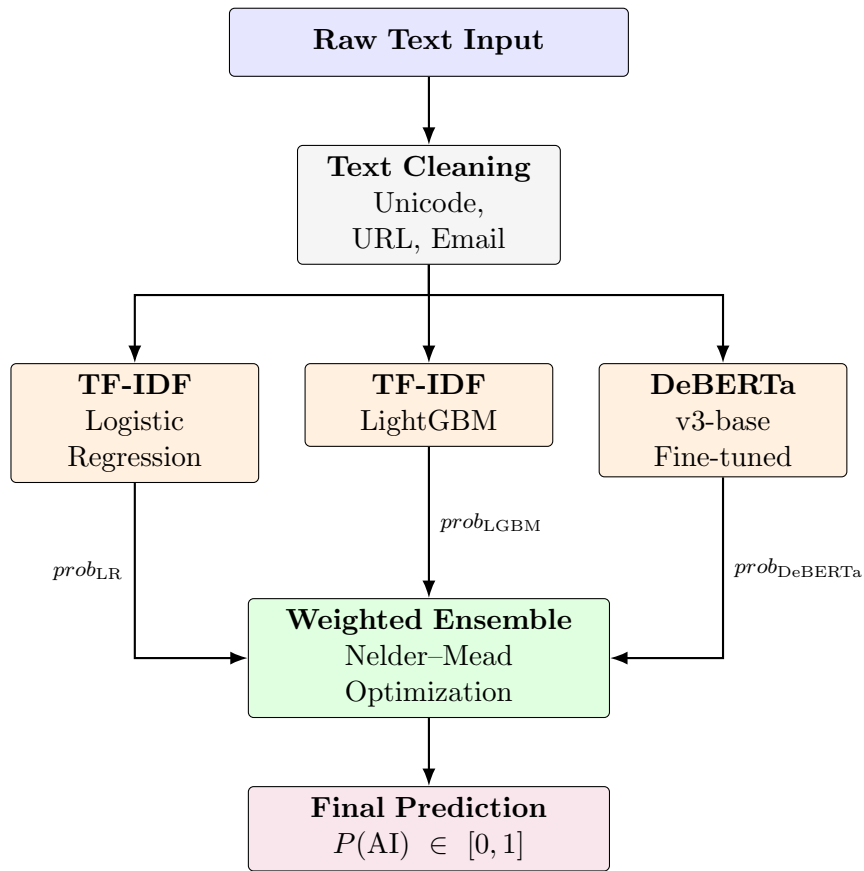
Nhãn	Ý nghĩa
0	Human-written: văn bản do con người viết
1	AI-generated: văn bản do mô hình ngôn ngữ lớn (LLM) sinh ra

Bảng 2: Quy ước nhãn trong toàn bộ báo cáo

1.2. Mục Tiêu

- Huấn luyện lại một mô hình phù hợp trên bộ dữ liệu đã chọn.
- Đánh giá mô hình bằng các độ đo phù hợp, đặc biệt **ROC-AUC** (metric chính của cuộc thi Kaggle).
- Xây dựng ứng dụng website demo sử dụng mô hình đã huấn luyện.

1.3. Kiến Trúc Tổng Thể Pipeline



Hình 1: Kiến trúc tổng thể của pipeline end-to-end

2. Cấu Trúc Thư Mục Và Vai Trò Từng Phần

```

Detect-AI-Generated-Text-Using-LLM/
├── data/
│   ├── train_v2_drcat_02.csv ..... Dataset gốc DAIGT-V2
│   ├── train_with_features.pkl ..... Dữ liệu gốc + feature EDA
│   └── train_clean.pkl ..... Dữ liệu sạch + fold + split
├── notebooks/
│   ├── task1_eda.ipynb ..... Khám phá dữ liệu
│   ├── task2_pipeline.ipynb ..... Tiền xử lý, split, tokenize
│   ├── ablation-deberta.ipynb ..... Fine-tune DeBERTa trên Kaggle
│   └── task4_ensemble.ipynb ..... Baseline, ensemble, đánh giá
├── outputs/
│   ├── checkpoints/ ..... Trọng số mô hình
│   │   └── deberta_tuned_best.pt ..... DeBERTa final tuned
│   ├── eda/ ..... Hình EDA
│   ├── ensemble/ ..... ROC/PR curves, kết quả ensemble
│   ├── logs/ ..... Log huấn luyện
│   ├── pipeline/ ..... Token length distribution
│   ├── tv2_deberta_tuning/ ..... Ablation DeBERTa
│   │   ├── report_training_process/ ..... Loss curve, validation metrics
│   │   ├── best_config.json ..... Cấu hình tốt nhất
│   │   └── best_tuned_test_metrics.csv ..... Metric test cuối cùng
├── src/
│   ├── modeling.py ..... DeBERTaDetector, MeanPooling
│   ├── evaluation.py ..... Metrics, ROC/PR/CM
│   └── inference.py ..... Pipeline inference CLI
├── web_ui/
│   └── app.py ..... Ứng dụng Gradio demo
├── requirements.txt ..... Danh sách thư viện Python
└── README.md ..... File README

```

Hình 2: Cấu trúc thư mục của dự án

2.1. Vai Trò Của Các Notebook Chính

Notebook	Vai trò	Đầu ra chính
task1_eda.ipynb	Khám phá dữ liệu, kiểm tra chất lượng, phân phối nhãn, đặc trưng văn bản	train_with_features.pkl, outputs/eda/fig*.png
task2_pipeline.ipynb	Tiền xử lý văn bản, tạo text_clean, chia train_val/test, tạo fold stratified	train_clean.pkl, token_length_distribution.png
ablation-deberta.ipynb	Fine-tune DeBERTa-v3-base trên Kaggle GPU, lựa chọn tham số, lưu checkpoint	deberta_tuned_best.pt, outputs/tv2-deberta_tuning/
task4_ensemble.ipynb	Huấn luyện TF-IDF baselines, load DeBERTa checkpoint, so sánh mô hình, ensemble	outputs/ensemble/

Bảng 3: Vai trò và đầu ra của các notebook chính

2.2. Thứ Tự Chạy Đúng

1. notebooks/task1_eda.ipynb	-> EDA + feature engineering
2. notebooks/task2_pipeline.ipynb	-> Data pipeline + train_clean.pkl
3. notebooks/ablation-deberta.ipynb	-> Fine-tune DeBERTa (trên Kaggle GPU)
4. notebooks/task4_ensemble.ipynb	-> Baselines + ensemble + danh gia

Mỗi task phụ thuộc vào artifact của task trước. Task 2 cần train_with_features.pkl từ Task 1. Task 3/ablation cần train_clean.pkl từ Task 2. Task 4 cần cả train_clean.pkl và checkpoint DeBERTa từ Task 3.

3. Mô tả dữ liệu

3.1. Nguồn Dữ Liệu

Dữ liệu chính được sử dụng là **DAIGT-V2** (Detect AI Generated Text Version 2), lấy từ cuộc thi Kaggle. File gốc:

```
data/train_v2_drcat_02.csv -> (44,868 mau x 5 cot)
```

Các cột trong dataset gốc:

Cột	Ý nghĩa
text	Nội dung văn bản (essay)
label	Nhân 0 (Human) hoặc 1 (AI)
prompt_name	Tên đề bài (prompt) dùng để sinh essay
source	Nguồn dữ liệu
RDizzl3_seven	Cờ đánh dấu phiên bản dataset

Bảng 4: Các cột trong dataset gốc `train_v2_drcat_02.csv`

Sau khi tiền xử lý và chia dữ liệu, notebook `task2_pipeline.ipynb` sinh ra:

```
data/train_clean.pkl    ->    (44,868 mau x 4 cot)
```

Cột	Ý nghĩa
text_clean	Văn bản sau tiền xử lý tối thiểu
label	Nhân 0/1 tương ứng Human/AI
split	Tập dữ liệu: <code>train_val</code> hoặc <code>test</code>
fold	Fold validation trong phần <code>train_val</code> ; test có <code>fold = -1</code>

Bảng 5: Các cột trong `train_clean.pkl` sau tiền xử lý

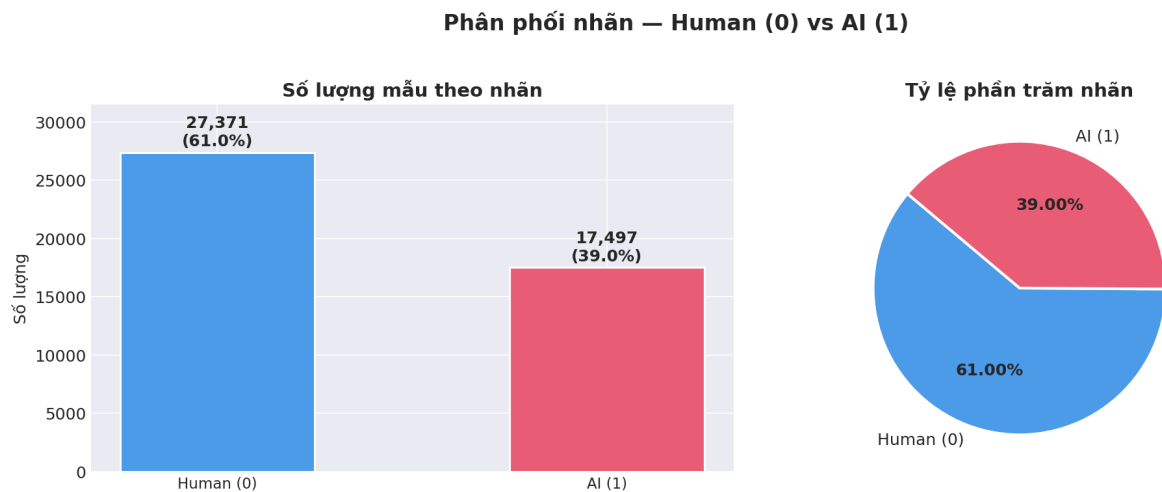
3.2. Phân Phối Nhãn

Tổng số mẫu trong `train_clean.pkl` là **44,868**.

Label	Ý nghĩa	Số mẫu	Tỷ lệ
0	Human-written	27,371	61.00%
1	AI-generated	17,497	39.00%

Bảng 6: Phân phối nhãn trong dataset DAIGT-V2

Dataset có **lệch nhãn ở mức vừa phải** (tỷ lệ $\approx 61:39$). Vì vậy, báo cáo không chỉ dùng accuracy mà còn sử dụng **precision**, **recall**, **F1**, **ROC-AUC** và **PR-AUC** để đánh giá toàn diện hơn.



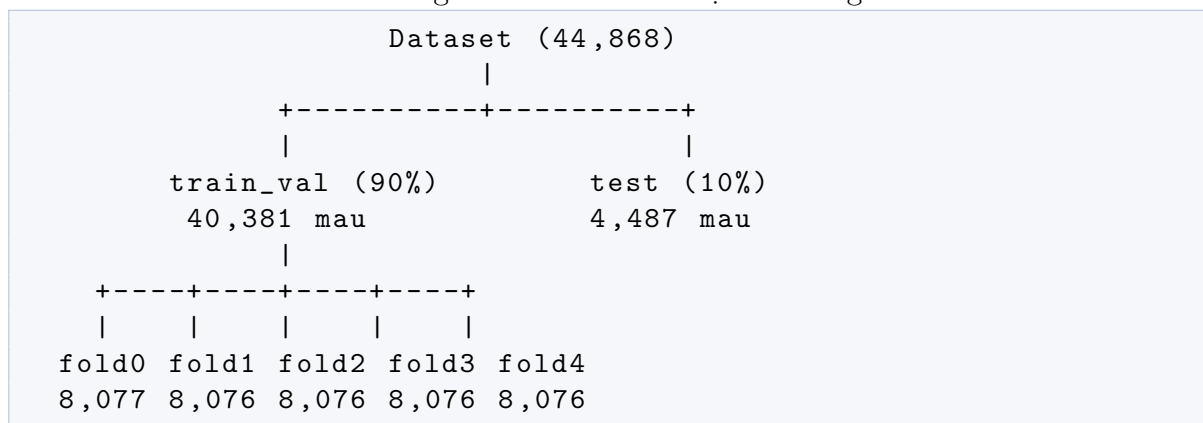
Hình 3: Phân phối nhãn Human/AI trong dataset DAIGT-V2

3.3. Chia Dữ Liệu

Dự án sử dụng chiến lược chia dữ liệu gồm **hai tầng**:

- Tầng 1:** Giữ riêng **10% dữ liệu** làm **held-out test set** — tập này chỉ dùng một lần duy nhất ở cuối để đánh giá mô hình đã chốt.
- Tầng 2:** Trên **90% còn lại** (train_val), tạo **Stratified K-Fold 5 fold** để chọn validation fold.

Listing 1: Sơ đồ chia dữ liệu hai tầng



Trong cấu hình huấn luyện hiện tại:

Tập	Điều kiện chọn	Số mẫu
Train	split == 'train_val' và fold != 0	32,304
Validation	split == 'train_val' và fold == 0	8,077
Test	split == 'test'	4,487

Bảng 7: Kích thước các tập dữ liệu sau khi chia

Phân phối nhãn theo split:

Split	Human (label=0)	AI (label=1)
train_val	24,634	15,747
test	2,737	1,750

Bảng 8: Phân phối nhãn theo split

Tại sao chia như vậy? Validation được dùng để chọn hyperparameter và early stopping, còn test set chỉ dùng ở cuối để đánh giá mô hình đã chốt. Đây là điểm quan trọng để tránh **optimistic bias** — nếu dùng test set để chọn mô hình, kết quả sẽ lạc quan hơn thực tế.

3.4. Duplicate Và Leakage Check

Trong quá trình xử lý, văn bản được chuẩn hóa nhẹ để kiểm tra duplicate:

```

1 def normalize_for_split(series):
2     return (
3         series.fillna('')
4         .astype(str)
5         .str.lower()
6         .str.replace(r'\s+', ' ', regex=True)
7         .str.strip()
8     )

```

Notebook ablation (ablation-deberta.ipynb) có bước loại các duplicate đã chuẩn hóa trước khi chia lại dữ liệu. Khi chạy trên Kaggle, log ghi nhận có **6 duplicate normalized** được loại bỏ trước khi split. Sau đó notebook kiểm tra **overlap giữa train, validation và test** — nếu phát hiện overlap khác 0, notebook dừng ngay để tránh data leakage.

4. Tiền xử lý Và Phân Tích Dữ Liệu

Notebook task1_edu.ipynb thực hiện các bước EDA chính:

1. Đọc dataset gốc train_v2_drcat_02.csv
2. Kiểm tra shape, columns, missing values và duplicate
3. Vẽ phân phối nhãn Human/AI
4. Tính các đặc trưng thống kê của văn bản
5. So sánh đặc trưng giữa hai lớp
6. Lưu hình vào outputs/eda/

4.1. Kiểm Tra Chất Lượng Dữ Liệu

Hàm `check_data_quality(df)` kiểm tra:

- **Missing values** theo cột
- **Duplicate rows** (toàn bộ dòng trùng lặp)

- **Empty strings** (văn bản rỗng)
- **Data types** của từng cột

Đây là bước quan trọng trước khi modeling. Nếu có text rỗng, label lỗi hoặc duplicate quá nhiều thì model có thể học sai hoặc bị leak.

4.2. Feature Engineering Cho EDA

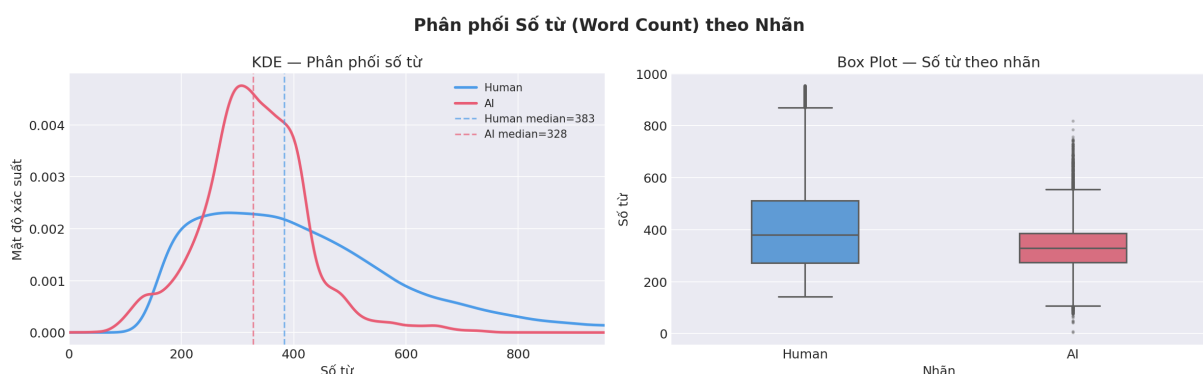
Hàm `compute_text_features` tạo các đặc trưng thống kê:

Feature	Ý nghĩa	Tại sao quan trọng với AI detection
<code>word_count</code>	Số từ trong văn bản	AI có thể sinh văn bản dài/đều hơn
<code>char_count</code>	Số ký tự	Tương quan với word count
<code>sent_count</code>	Số câu (tách theo ., !, ?)	AI thường có số câu đều đặn
<code>avg_word_len</code>	Độ dài trung bình của từ	AI có thể dùng từ phức tạp hơn
<code>unique_words</code>	Số từ duy nhất (types)	—
<code>lexical_diversity</code>	Tỷ lệ <code>unique_words</code> / <code>word_count</code> (TTR)	AI có thể có TTR thấp hơn do lặp pattern
<code>punct_count</code>	Số dấu câu	AI thường dùng dấu câu đều đặn
<code>digit_count</code>	Số chữ số	—
<code>upper_ratio</code>	Tỷ lệ ký tự viết hoa	Human có thể viết hoa tự do hơn
<code>words_per_sent</code>	Số từ trung bình mỗi câu	AI thường có câu dài, đều

Bảng 9: Các đặc trưng thống kê văn bản được tính trong EDA

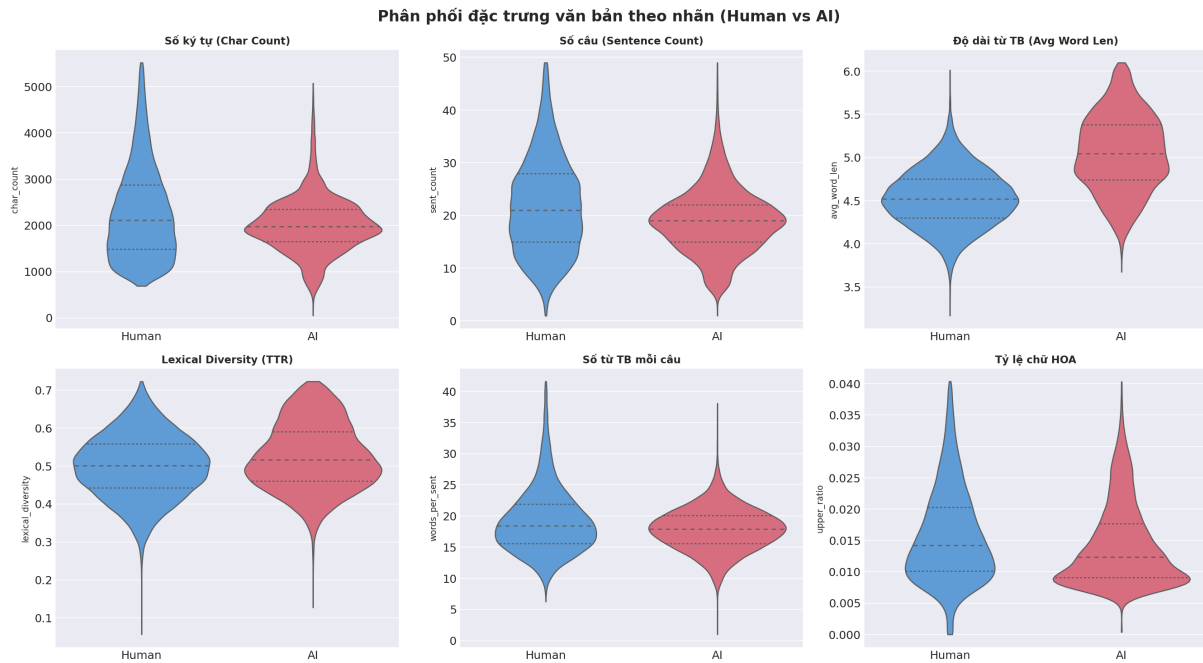
4.3. Các Hình EDA Chính

Phân phối số từ theo nhãn:



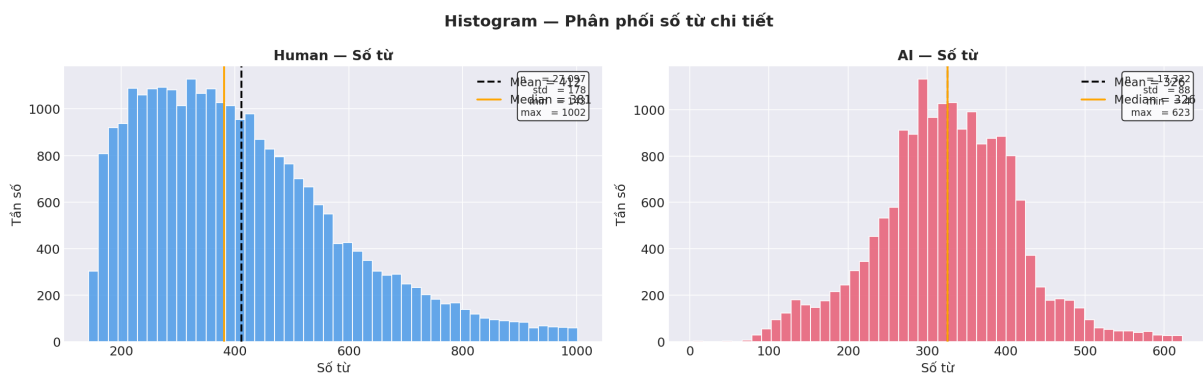
Hình 4: Phân phối số từ (word count) theo nhãn Human/AI

Phân phối các feature thống kê:



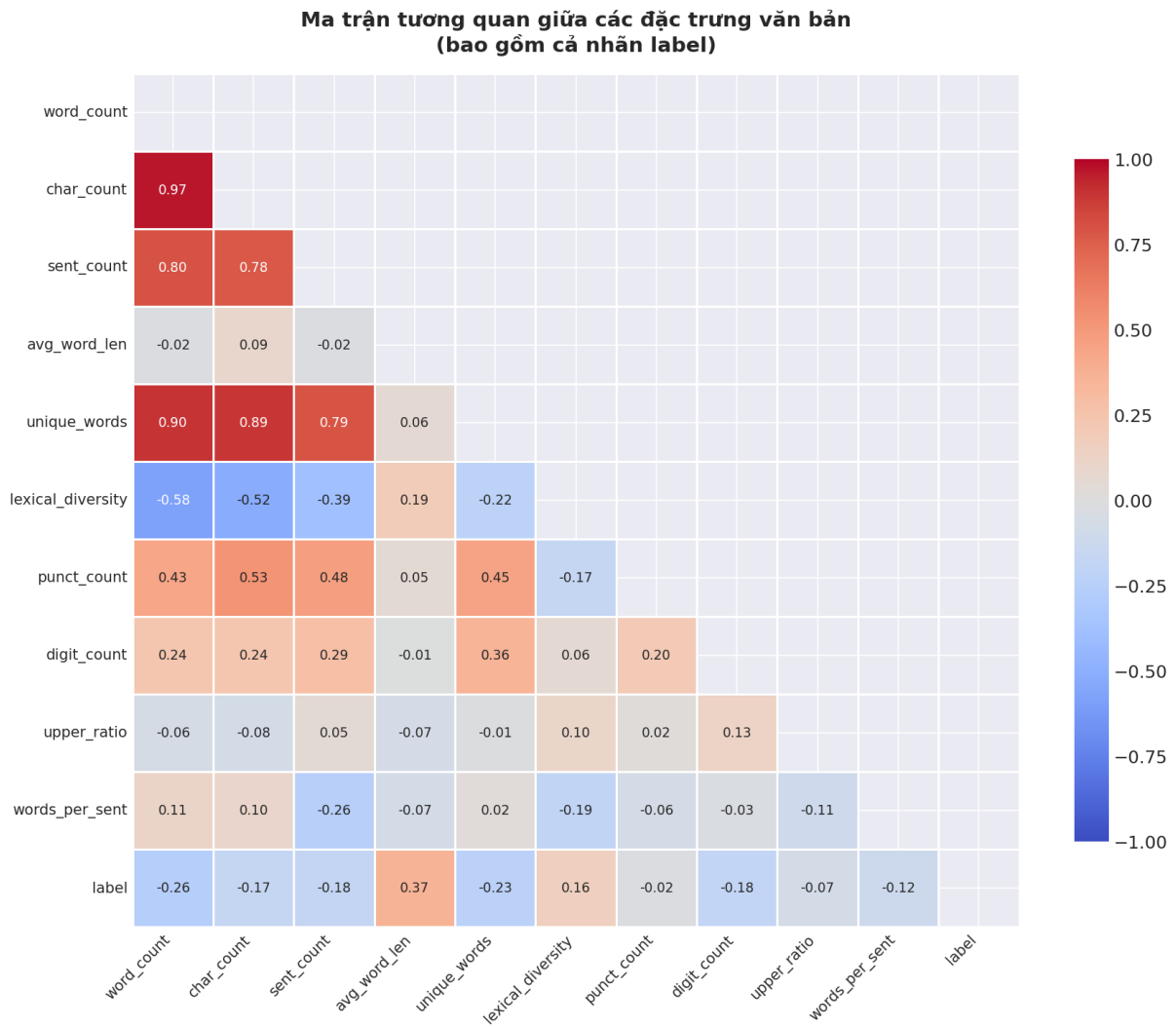
Hình 5: Phân phối các đặc trưng thống kê: word count, char count, lexical diversity, v.v.

Histogram word count chi tiết:



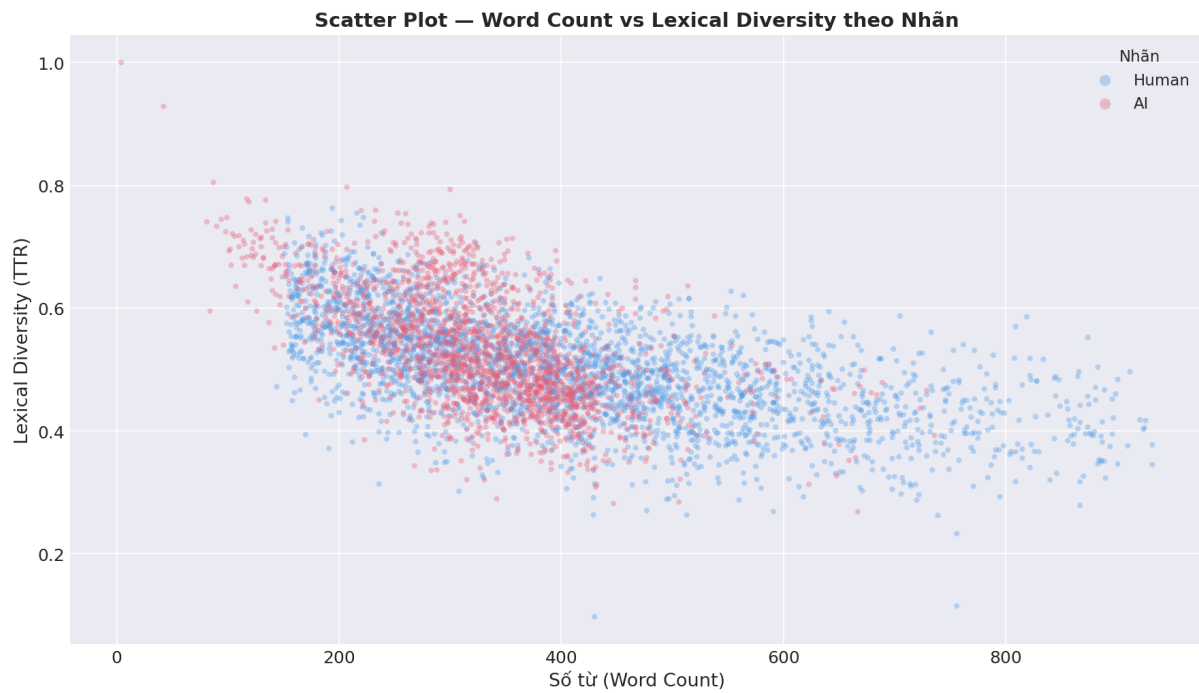
Hình 6: Histogram chi tiết phân phối số từ theo từng lớp

Correlation heatmap:



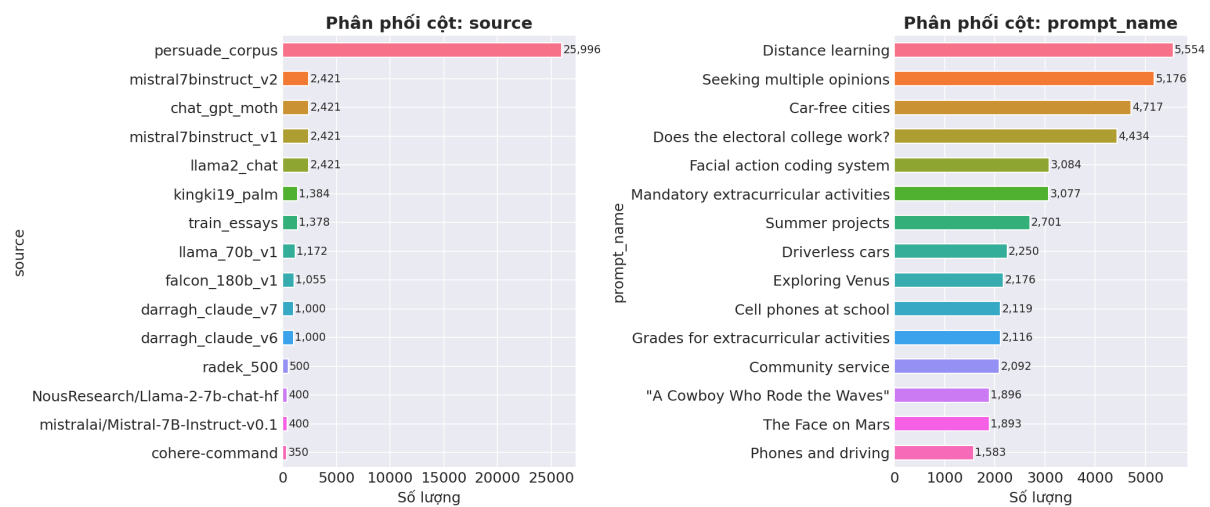
Hình 7: Ma trận tương quan giữa các feature thống kê và label

Scatter: Word count vs. Lexical diversity:



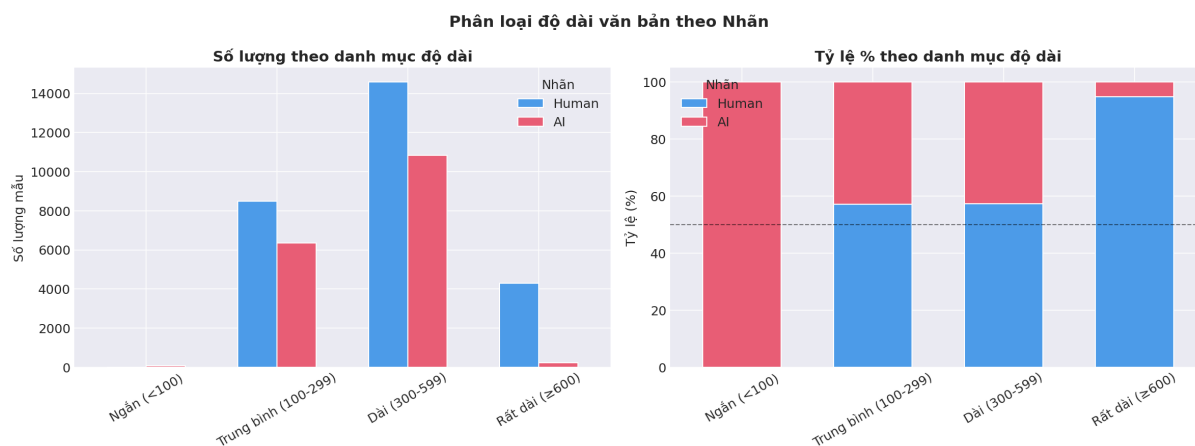
Hình 8: Quan hệ giữa word count và lexical diversity theo nhãn

Phân phối metadata:



Hình 9: Phân phối metadata: source, prompt_name

Nhóm độ dài văn bản:



Hình 10: Phân phối nhóm văn bản ngắn/trung bình/dài theo nhân

4.4. Nhận Xét Từ EDA

- Các đặc trưng bề mặt như **độ dài**, **cấu trúc câu**, **lexical diversity** và **dấu câu** có sự khác biệt nhất định giữa Human và AI.
- Tuy nhiên, các đặc trưng này chỉ hỗ trợ phân tích khám phá; mô hình chính vẫn cần học biểu diễn ngôn ngữ sâu hơn bằng Transformer.
- **Lexical diversity** của AI-generated text có xu hướng thấp hơn, có thể do AI lặp lại các pattern ngôn ngữ.
- **Word count** của AI text thường tập trung hơn quanh một giá trị trung bình, trong khi Human text phân tán hơn.

Notebook task2_pipeline.ipynb tạo pipeline chuẩn bị dữ liệu cho mô hình.

4.5. Các Bước Tiền Xử Lý

Bước	Mô tả	Lý do
Normalize Unicode	Đổi smart quotes, dash, ellipsis, non-breaking space, zero-width char	Giảm nhiễu ký tự lạ không mang thông tin
Replace URL	URL → [URL]	Giữ thông tin “có URL” nhưng không để link dài làm nhiễu
Replace email	Email → [EMAIL]	Bảo vệ thông tin cá nhân và giảm nhiễu
Normalize whitespace	Nhiều space/tab → 1 space; 3+ newline → 2 newline	Ổn định input

Bảng 10: Các bước tiền xử lý văn bản trong TextCleaner

Class TextCleaner compile regex pattern một lần duy nhất trong `__init__` để tránh recompile mỗi lần gọi `clean()`.

4.6. Tokenization

Tokenization được thực hiện bằng tokenizer của DeBERTa (SentencePiece BPE):

```
1 from transformers import AutoTokenizer
2
3 tokenizer = AutoTokenizer.from_pretrained('microsoft/deberta-
4 v3-base')
5
6 encoding = tokenizer(
7     text,
8     max_length=512,
9     padding='max_length',
10    truncation=True,
11    return_tensors='pt',
12 )
```

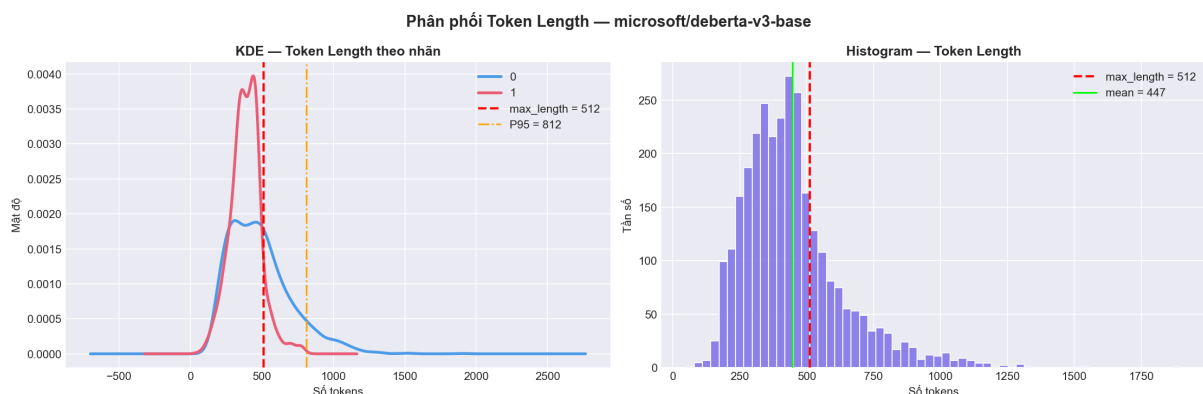
Các tensor đầu ra:

Tensor	Shape	Ý nghĩa
input_ids	[1, 512]	ID token đầu vào theo vocabulary
attention_mask	[1, 512]	1 cho token thật, 0 cho padding
labels	scalar	Nhãn 0/1

Bảng 11: Các tensor đầu ra sau tokenization

4.7. Phân Tích Token Length

Notebook sample tối đa 3,000 text để phân tích phân phối token length:



Hình 11: Phân phối token length sau khi tokenize bằng DeBERTa tokenizer

Nếu nhiều text bị truncate (dài hơn 512 tokens), model có thể mất thông tin cuối bài. Nếu tăng max_length, cần nhiều VRAM hơn.

4.8. Dataset Và DataLoader

Class AITextDataset biến text thành tensor sẵn sàng cho model:

```

1 class AITextDataset(Dataset):
2     def __getitem__(self, idx):
3         text = self.texts[idx]
4         encoding = self.tokenizer(
5             text,
6             max_length=self.max_length,
7             padding='max_length',
8             truncation=True,
9             return_tensors='pt',
10        )
11        item = {k: v.squeeze(0) for k, v in encoding.items()}
12        if self.labels is not None:
13            item['labels'] = torch.tensor(self.labels[idx],
14                                         dtype=torch.long)
15        return item

```

Dataloader gom nhiều sample thành batch:

- **Train loader:** shuffle=True để tránh model học theo thứ tự
- **Valid loader:** shuffle=False để kết quả ổn định
- pin_memory=True nếu có CUDA, tăng tốc copy CPU → GPU
- num_workers=0 trên Windows để tránh lỗi multiprocessing

4.9. Validate Pipeline

Hàm validate_pipeline kiểm tra nhanh:

- input_ids không âm
- attention_mask chỉ gồm 0/1
- labels chỉ gồm 0/1
- Tensor move được sang device (CPU/GPU)
- Shape đúng

Đây là bước nhỏ nhưng rất có ích, vì bug tokenizer/dataloader thường chỉ lộ ra khi train.

5. Mô Hình Baseline

Notebook task4_ensemble.ipynb huấn luyện các baseline truyền thống để so sánh với Transformer.

5.1. TF-IDF + Logistic Regression

Baseline đầu tiên dùng **TF-IDF** ở cả **word-level** và **char-level**, sau đó ghép đặc trưng sparse:

Vectorizer	Cấu hình	Bắt pattern gì
Word TF-IDF	Unigram + bigram; max 100,000 features	Từ và cụm từ
Char TF-IDF	char_wb 3-6 gram; max 100,000 features	Pattern ký tự, dấu câu, tiền/hậu tố, phong cách viết

Bảng 12: Cấu hình TF-IDF vectorizer

```

1 from scipy.sparse import hstack
2
3 X = hstack([X_word, X_char])    # Ghep feature sparse

```

Logistic Regression được cấu hình:

```

1 LogisticRegression(C=5.0, max_iter=2000, solver='saga')

```

Tại sao baseline này mạnh:

- TF-IDF bắt được từ/cụm từ hay xuất hiện trong AI text (ví dụ: “in conclusion”, “it is important to note”).
- Char n-gram bắt được **style bề mặt** — cách dùng dấu câu, viết hoa, cấu trúc ký tự đặc trưng.
- Logistic Regression nhanh, ổn định, dễ giải thích.

5.2. TF-IDF + SGDClassifier

SGDClassifier là baseline tuyến tính khác, huấn luyện nhanh bằng **Stochastic Gradient Descent**. Phù hợp với dữ liệu sparse và dùng để tạo thêm một góc nhìn baseline nhẹ.

5.3. TF-IDF + LightGBM

LightGBM được dùng trên TF-IDF features để **học các quan hệ phi tuyến** mà Logistic Regression bỏ qua.

Cấu hình đáng chú ý:

```

1 lgb.LGBMClassifier(
2     objective='binary',
3     metric='auc',
4     learning_rate=0.05,
5     n_estimators=1000,
6     num_leaves=63,
7     feature_fraction=0.3,
8     bagging_fraction=0.8,
9 )

```

Dù vẫn bị giới hạn bởi biểu diễn bag-of-words/char-ngram, LightGBM thường mạnh khi có nhiều feature sparse và đủ dữ liệu.

6. Mô Hình Chính: DeBERTa-v3-base

6.1. Tổng Quan DeBERTa

DeBERTa (Decoding-enhanced BERT with disentangled attention) được Microsoft phát triển với hai cải tiến chính so với BERT/RoBERTa:

6.1.1. a) *Disentangled Attention (Chú Ý Phân Ly)*

Trong BERT, mỗi token được biểu diễn bởi **một vector duy nhất** kết hợp cả nội dung và vị trí. DeBERTa tách biệt thành **hai vector riêng biệt**:

$$A_{i,j} = \underbrace{H_i W_q (H_j W_k)^T}_{\text{content-to-content}} + \underbrace{H_i W_q (P_{i|j} W_k)^T}_{\text{content-to-position}} + \underbrace{P_{j|i} W_q (H_j W_k)^T}_{\text{position-to-content}} \quad (1)$$

Trong đó:

- H_i : Vector biểu diễn nội dung token thứ i
- $P_{i|j}$: Vector biểu diễn vị trí tương đối của i so với j

Lợi ích cho AI Text Detection: AI thường có cấu trúc câu và pattern vị trí từ khác con người; việc phân tách nội dung-vị trí giúp DeBERTa nhận diện tốt hơn.

6.1.2. b) *Enhanced Mask Decoder (EMD)*

DeBERTa sử dụng absolute position embedding chỉ tại bước softmax cuối (thay vì đầu vào như BERT), giúp mô hình học position tốt hơn trong quá trình pre-training.

6.1.3. c) *DeBERTa-v3: RTGM + SiFT*

DeBERTa-v3 sử dụng kỹ thuật **Replaced Token Detection** (từ ELECTRA) và **Scale-invariant Fine-tuning (SiFT)** để cải thiện khả năng fine-tuning trên downstream tasks.

6.2. Cấu Hình Mô Hình

Thông số	Giá trị
Backbone	microsoft/deberta-v3-base
Hidden size	768
Attention heads	12
Transformer layers	12
Max position embeddings	512
Vocab size	128,100
Tổng tham số	$\approx 183\text{M}$
Pooling strategy	Mean Pooling qua attention mask
Classifier	Linear (768, 2) với Dropout(0.1)

Bảng 13: Cấu hình mô hình DeBERTa-v3-base

6.3. Pipeline Mô Hình

Listing 2: Pipeline xử lý của DeBERTaDetector

```

input_ids + attention_mask
    |
DeBERTa-v3-base backbone
    |
last_hidden_state [batch, seq_len, 768]
    |
Mean Pooling theo attention_mask
    |
Dropout(0.1)
    |
Linear(768, 2)
    |
logits [batch, 2]
    |
softmax -> P(Human), P(AI)

```

6.4. Mean Pooling

Mean Pooling lấy **trung bình** hidden states của các token thật, bỏ qua padding token bằng `attention_mask`:

```

1 class MeanPooling(nn.Module):
2     """Mean pooling over non-padding tokens."""
3
4     def forward(self, hidden_state, attention_mask):
5         mask = attention_mask.unsqueeze(-1).float()
6         summed = (hidden_state * mask).sum(dim=1)
7         counts = mask.sum(dim=1).clamp(min=1e-9)
8         return summed / counts

```

Tại sao dùng Mean Pooling thay vì CLS token? Mean Pooling phù hợp với essay-level classification vì nhãn thuộc về **toàn bộ đoạn văn**, không chỉ token đầu tiên. Nó tạo biểu diễn đại diện cho toàn bộ chuỗi văn bản.

6.5. Classifier Head

Phần classifier gồm dropout và linear layer:

```

1 class DeBERTaDetector(nn.Module):
2     def forward(self, input_ids, attention_mask,
3         token_type_ids=None):
4         outputs = self.backbone(
5             input_ids=input_ids,
6             attention_mask=attention_mask,
7             token_type_ids=token_type_ids,
8         )
9         if self.pooling == "cls":
10             pooled = outputs.last_hidden_state[:, 0]

```

```
10         else:
11             pooled = self.mean_pooler(outputs.
12                                     last_hidden_state, attention_mask)
13             return self.classifier(self.dropout(pooled))
```

Với `num_labels = 2`, logits có hai cột:

- **Cột 0:** điểm cho Human
- **Cột 1:** điểm cho AI

Khi inference, xác suất AI được lấy bằng:

```
1 prob_ai = torch.softmax(logits.float(), dim=-1)[: , 1]
```

i Ghi chú: Việc ép `logits.float()` trước `softmax` giúp tránh lỗi precision khi dùng AMP hoặc checkpoint có dtype không đồng nhất (FP16 logits có thể gây NaN trong softmax).

7. Fine-tuning Và Lựa Chọn Hyperparameter

Notebook `ablation-deberta.ipynb` được dùng để fine-tune DeBERTa và chọn cấu hình cuối cùng. Vì giới hạn thời gian GPU trên Kaggle (≈ 2 GPU hours), chiến lược được chuyển từ grid search lớn sang **tuning ít nhưng có định hướng**.

7.1. Cấu Hình Cuối Cùng (Final)

Tham số	Giá trị	Ý nghĩa
Backbone	microsoft/deberta-v3-base	Mô hình nền tảng
max_length	512	Độ dài tối đa sau tokenize
pooling	mean	Chiến lược pooling
dropout	0.1	Tỷ lệ dropout
lr_backbone	2e-5	Learning rate cho backbone
lr_head	1e-4	Learning rate cho classifier head
llrd_factor	0.9	Hệ số decay layerwise
weight_decay	0.01	Regularization
warmup_ratio	0.1	10% bước đầu warmup
scheduler_type	cosine	Kiểu giảm learning rate
batch_size	8	Batch size vật lý
grad_accum	4	Gradient accumulation steps
Effective batch size	32	$= 8 \times 4$
epochs	3	Số epoch huấn luyện
use_amp	False	Tắt AMP (tránh lỗi FP16)
label_smoothing	0.0	Không dùng label smoothing
use_class_weights	False	Không cân trọng số lớp
patience	1	Early stopping patience
primary_metric	auc	Metric chọn checkpoint

Bảng 14: Cấu hình fine-tuning DeBERTa cuối cùng

Checkpoint final được lưu tại: outputs/checkpoints/deberta_tuned_best.pt

Config tốt nhất: outputs/tv2_deberta_tuning/best_config.json

7.2. Optimizer Và Layerwise Learning Rate Decay (LLRD)

Mô hình dùng **AdamW** với **Layerwise Learning Rate Decay**. Ý tưởng: các layer thấp (gần embedding) của DeBERTa đã học các đặc trưng cú pháp cơ bản rất tốt từ pre-training, nên cần **learning rate nhỏ hơn** để tránh catastrophic forgetting. Classifier head mới hoàn toàn nên được học nhanh hơn.

Công thức tổng quát:

$$\text{lr}_{\text{layer } l} = \text{lr}_{\text{backbone}} \times \alpha^{L-l} \quad (2)$$

Trong đó $\alpha = 0.9$ (llrd_factor), L là tổng số layers, l là layer hiện tại.

```

Classifier head: lr = 1e-4 (cao nhất)
Layer 11 (top):  lr = 2e-5 * 0.9^0 = 2.00e-5
Layer 10:       lr = 2e-5 * 0.9^1 = 1.80e-5
Layer 9:        lr = 2e-5 * 0.9^2 = 1.62e-5
...
Layer 0 (bottom): lr = 2e-5 * 0.9^11 ~ 6.28e-6 (thấp nhất)

```

```
Embedding:      lr = rat  nho
```

Optimizer cũng **tách weight decay**: áp dụng cho weight bình thường, nhưng **không áp dụng** cho bias, LayerNorm.weight, LayerNorm.bias.

7.3. Scheduler Và Warmup

Scheduler **cosine với warmup** giúp training ổn định:

$$\text{lr}(t) = \begin{cases} \text{lr}_{\max} \cdot \frac{t}{T_{\text{warmup}}} & t < T_{\text{warmup}} \\ \frac{\text{lr}_{\max}}{2} \left(1 + \cos \frac{\pi(t - T_{\text{warmup}})}{T_{\text{total}} - T_{\text{warmup}}} \right) & t \geq T_{\text{warmup}} \end{cases} \quad (3)$$

- **10% bước đầu**: tăng learning rate dần (warmup)
- **Các bước sau**: giảm theo đường cosine

Điều này giúp tránh cập nhật quá mạnh ở đầu quá trình fine-tuning.

7.4. Training Loop

Training loop có các kỹ thuật quan trọng:

Kỹ thuật	Mô tả
Gradient accumulation	Tích lũy gradient qua 4 mini-batches trước khi cập nhật, tạo effective batch size = 32
Gradient clipping	<code>clip_grad_norm_(model.parameters(), max_grad_norm=1.0)</code> tránh gradient exploding
Early stopping	Dừng nếu validation AUC không cải thiện sau patience epoch
Checkpoint	Lưu <code>state_dict</code> khi validation AUC cải thiện

Bảng 15: Các kỹ thuật trong training loop

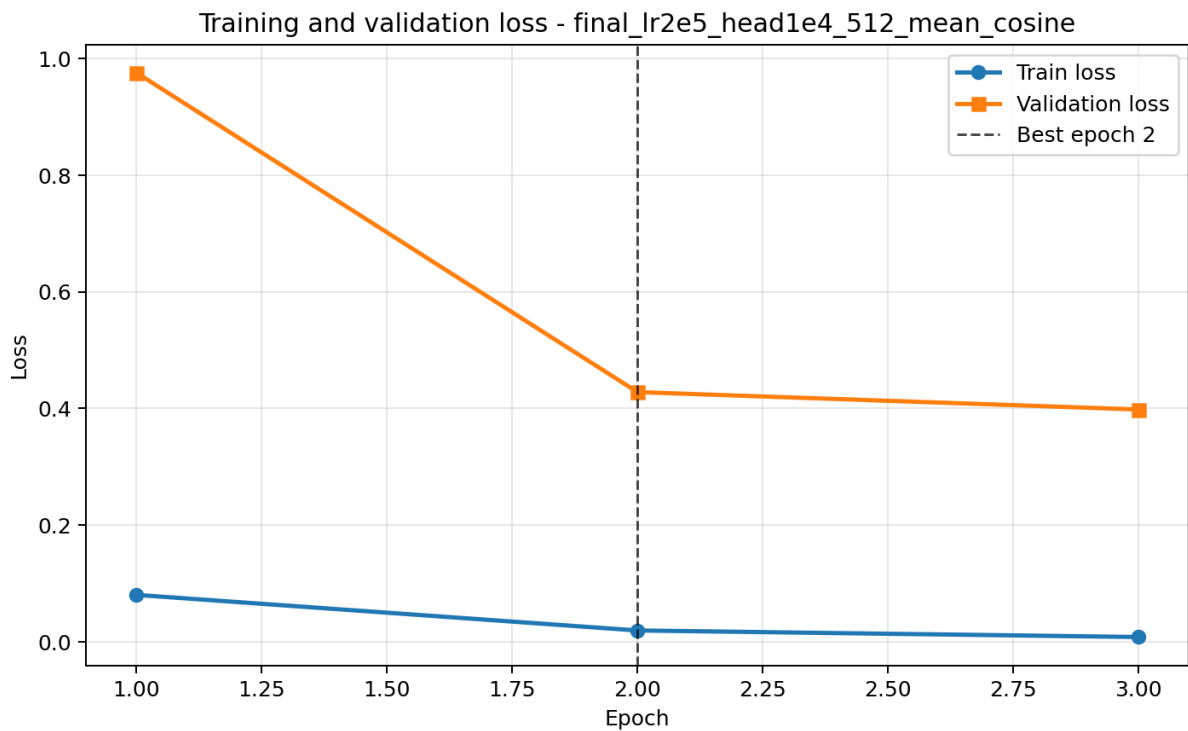
7.5. Training Curve

Kết quả log theo epoch của cấu hình final:

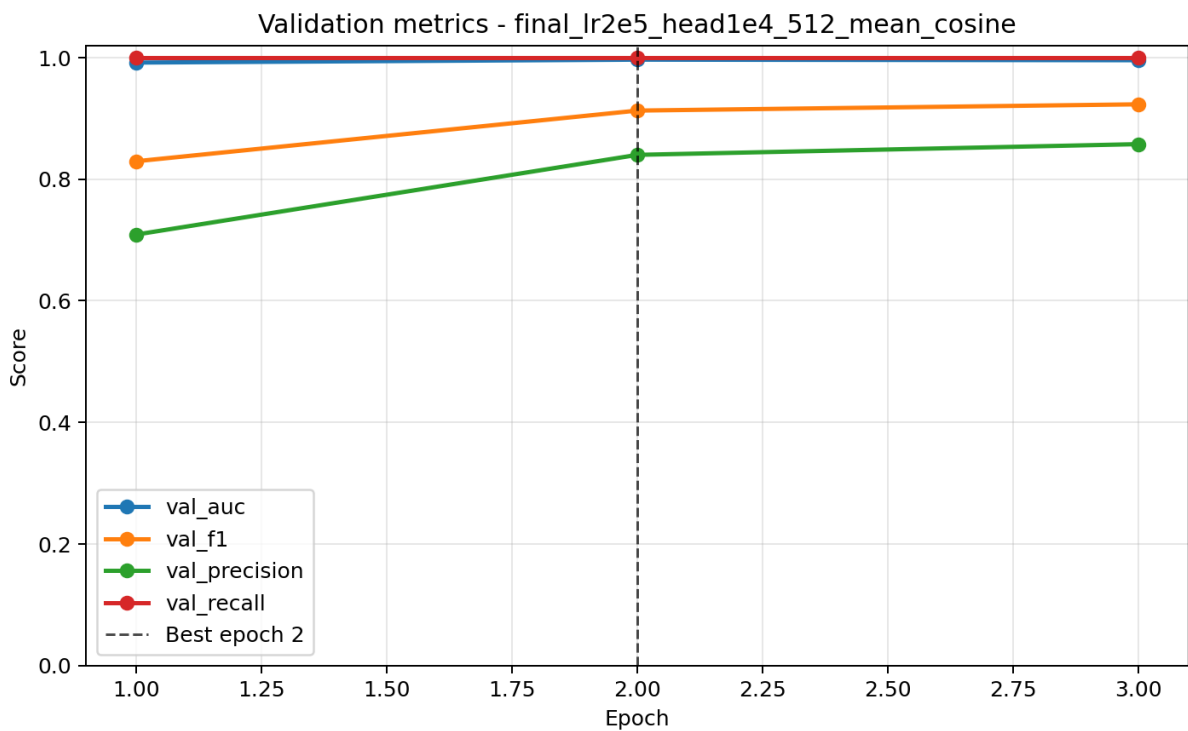
Epoch	Train Loss	Val Loss	Val AUC	Val F1	Val Prec.	Val Recall	Val Acc.
1	0.0800	0.9758	0.9917	0.8294	0.7088	0.9994	0.8396
2	0.0189	0.4278	0.9965	0.9127	0.8399	0.9994	0.9254
3	0.0077	0.3980	0.9955	0.9230	0.8575	0.9994	0.9350

Bảng 16: Log training theo epoch — Best epoch = 2 (Val AUC = 0.9965)

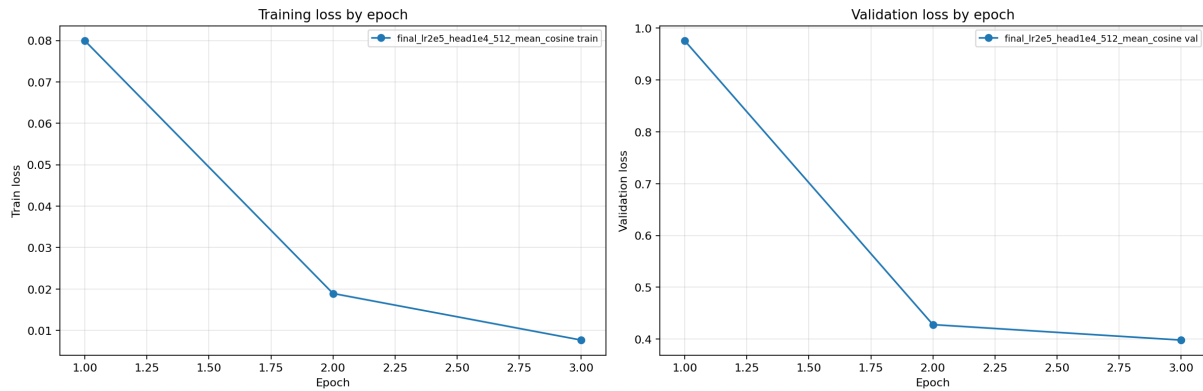
Ghi chú: **Best epoch = 2** (Val AUC = 0.9965 — cao nhất). Checkpoint của epoch 2 được giữ làm checkpoint tốt nhất.



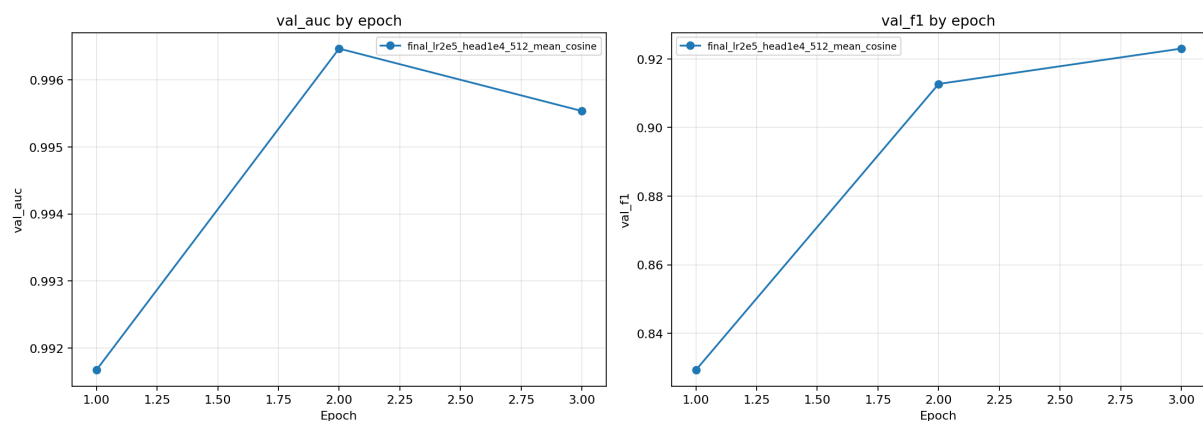
Hình 12: Training/Validation loss curve của cấu hình DeBERTa tốt nhất



Hình 13: Validation metrics (AUC, F1, Precision, Recall) theo epoch



Hình 14: Loss curves của tất cả các cấu hình trong ablation study



Hình 15: Metric curves của tất cả các cấu hình trong ablation study

7.6. Nhận Xét Training Curve

- **Train loss** giảm liên tục và rất nhanh ($0.08 \rightarrow 0.008$), cho thấy model học tốt trên training data.
- **Validation loss** giảm mạnh từ epoch 1 (0.976) sang epoch 2 (0.428), nhưng chỉ giảm nhẹ ở epoch 3 (0.398).
- **Val AUC** đạt đỉnh ở epoch 2 (0.9965), epoch 3 giảm nhẹ xuống 0.9955 — dấu hiệu bắt đầu overfitting.
- **Val Recall** cực cao (≈ 0.999) ở cả 3 epoch, nhưng **Val Precision** tăng dần ($0.709 \rightarrow 0.840 \rightarrow 0.857$), cho thấy model dần giảm false positive.
- Vì mục tiêu chọn model dùng ROC-AUC làm metric chính, checkpoint epoch 2 được giữ.
- Thời gian huấn luyện toàn bộ 3 epoch: **≈ 164.5 phút** trên Kaggle GPU.

8. Đánh Giá DeBERTa Trên Held-out Test Set và so sánh mô hình

Sau khi chọn cấu hình, mô hình được đánh giá **một lần duy nhất** trên held-out test set (4,487 mẫu). Đây là tập **không dùng** để chọn hyperparameter.

Metric	Giá trị	Nhận xét
Test Loss	0.4153	Tương đương val loss, model không quá overfit
Accuracy	0.9285	Đúng $\approx 93\%$
Precision	0.8453	Trong các dự đoán AI, 84.5% đúng
Recall	0.9994	Bắt được 99.9% văn bản AI
F1	0.9159	Cân bằng precision-recall
ROC-AUC	0.9982	Rất cao — xếp hạng xác suất tốt
PR-AUC	0.9935	Cao — precision-recall curve tốt

Bảng 17: Kết quả đánh giá DeBERTa tuned trên held-out test set

8.1. Nhận Xét

- **ROC-AUC = 0.998** rất cao, cho thấy mô hình **xếp hạng** xác suất Human/AI rất tốt.
- **Recall ≈ 1.0** nghĩa là mô hình **bắt được hầu hết** văn bản AI — rất ít false negative.
- **Precision = 0.845** thấp hơn recall đáng kể, cho thấy mô hình có xu hướng **dự đoán AI mạnh**; một số văn bản Human bị gán nhầm là AI (false positive). Điều này cần lưu ý trong web demo vì **false positive với văn bản người viết là rủi ro quan trọng** trong bối cảnh đánh giá bài viết thật.

8.2. Validation Results

Kết quả trên validation fold (8,077 mẫu), trích từ outputs/ensemble/model_comparison_metrics.csv:

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Weighted Ensemble	0.9950	0.9974	0.9898	0.9936	0.9997	0.9996
LR (TF-IDF)	0.9960	0.9981	0.9917	0.9949	0.9996	0.9995
LightGBM (TF-IDF)	0.9950	0.9974	0.9898	0.9936	0.9996	0.9995
SGD (TF-IDF)	0.9958	0.9971	0.9921	0.9946	0.9996	0.9994
Simple Average	0.9968	0.9984	0.9933	0.9959	0.9993	0.9992
DeBERTa-v3-base	0.9627	0.9162	0.9956	0.9542	0.9972	0.9954

Bảng 18: So sánh mô hình trên validation fold

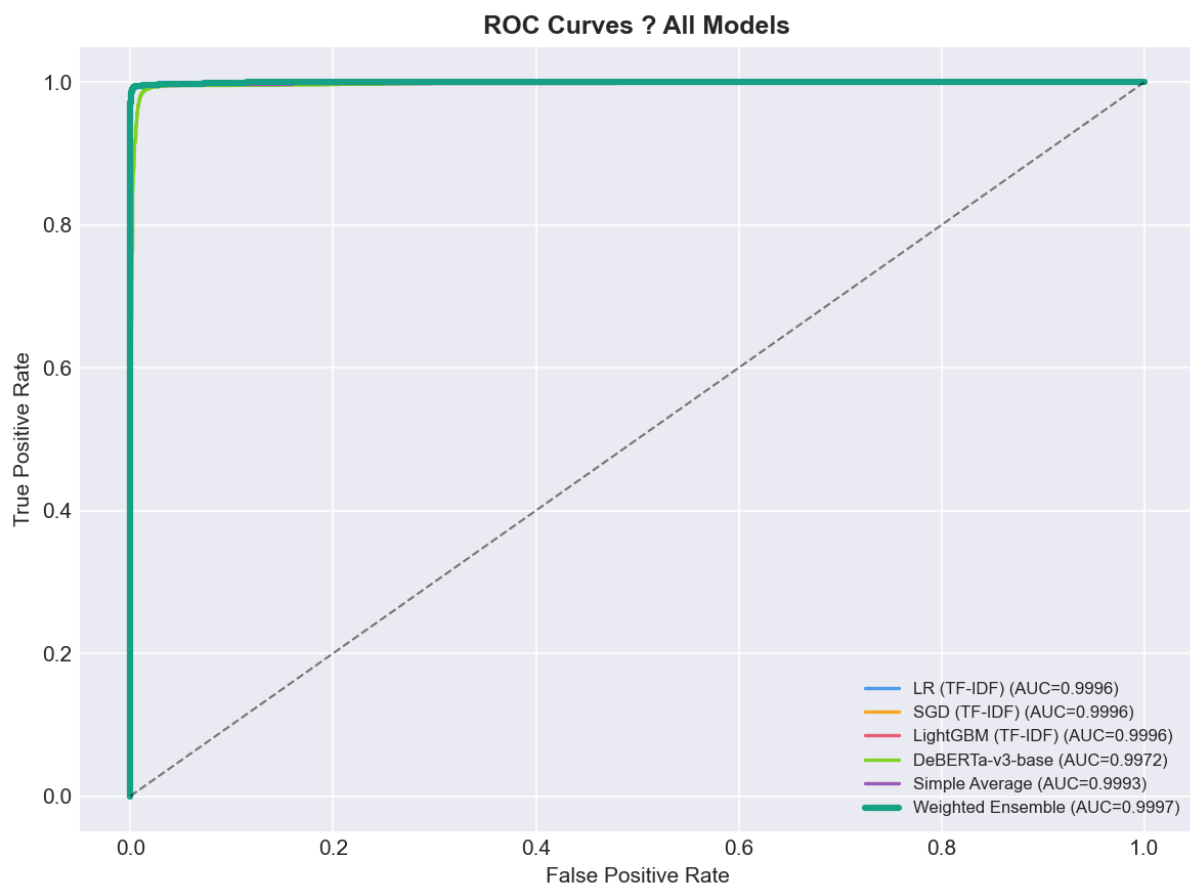
8.3. Test Results

Kết quả trên held-out test set (4,487 mẫu), trích từ outputs/ensemble/model_comparison_metrics_test

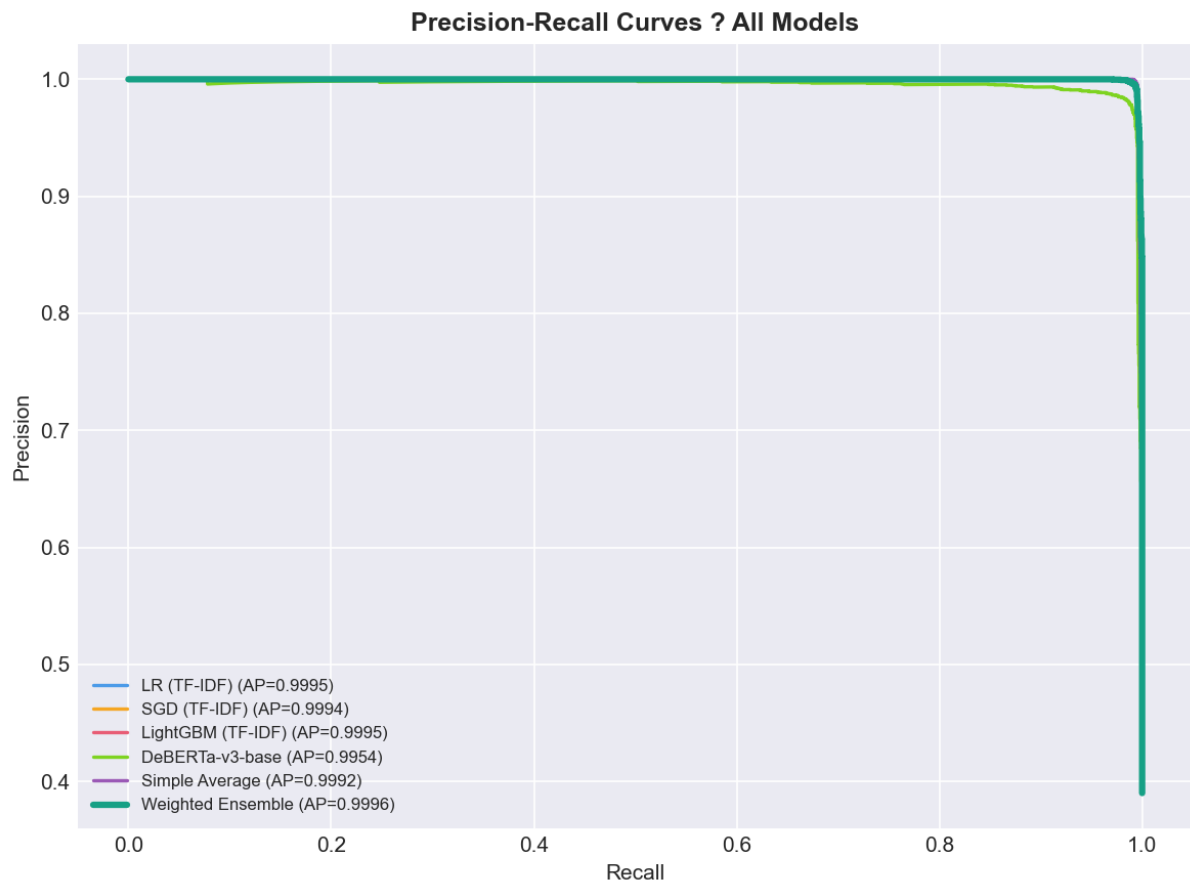
Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Simple Average	0.9962	0.9983	0.9920	0.9951	0.9997	0.9996
LR (TF-IDF)	0.9953	0.9994	0.9886	0.9940	0.9996	0.9996
Weighted Ensemble	0.9942	0.9983	0.9869	0.9925	0.9995	0.9994
SGD (TF-IDF)	0.9947	0.9977	0.9886	0.9931	0.9995	0.9995
LightGBM (TF-IDF)	0.9940	0.9983	0.9863	0.9922	0.9992	0.9991
DeBERTa-v3-base	0.9635	0.9191	0.9937	0.9550	0.9980	0.9971

Bảng 19: So sánh mô hình trên held-out test set

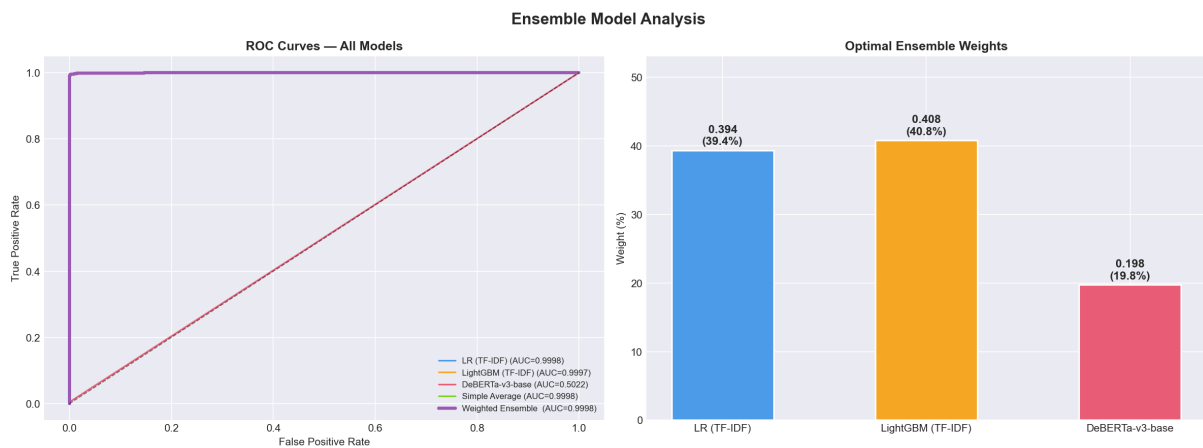
8.4. Hình Biểu Đồ So Sánh



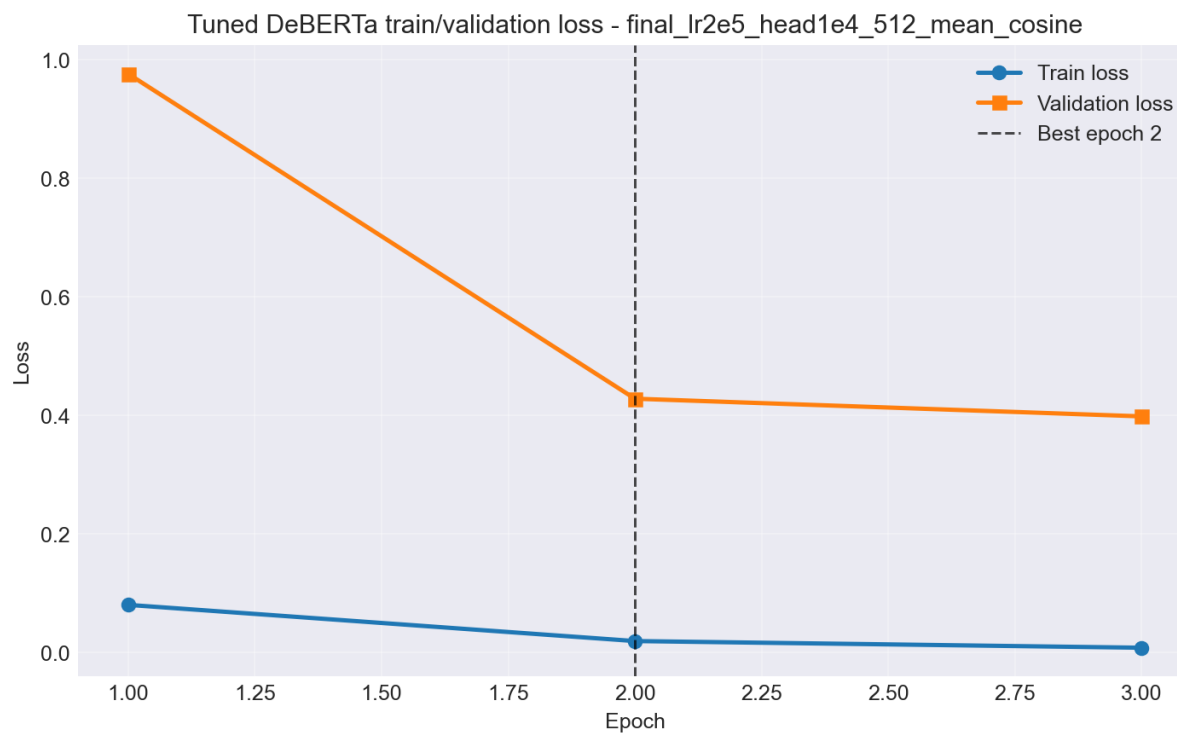
Hình 16: ROC curves so sánh tất cả các mô hình trên validation set



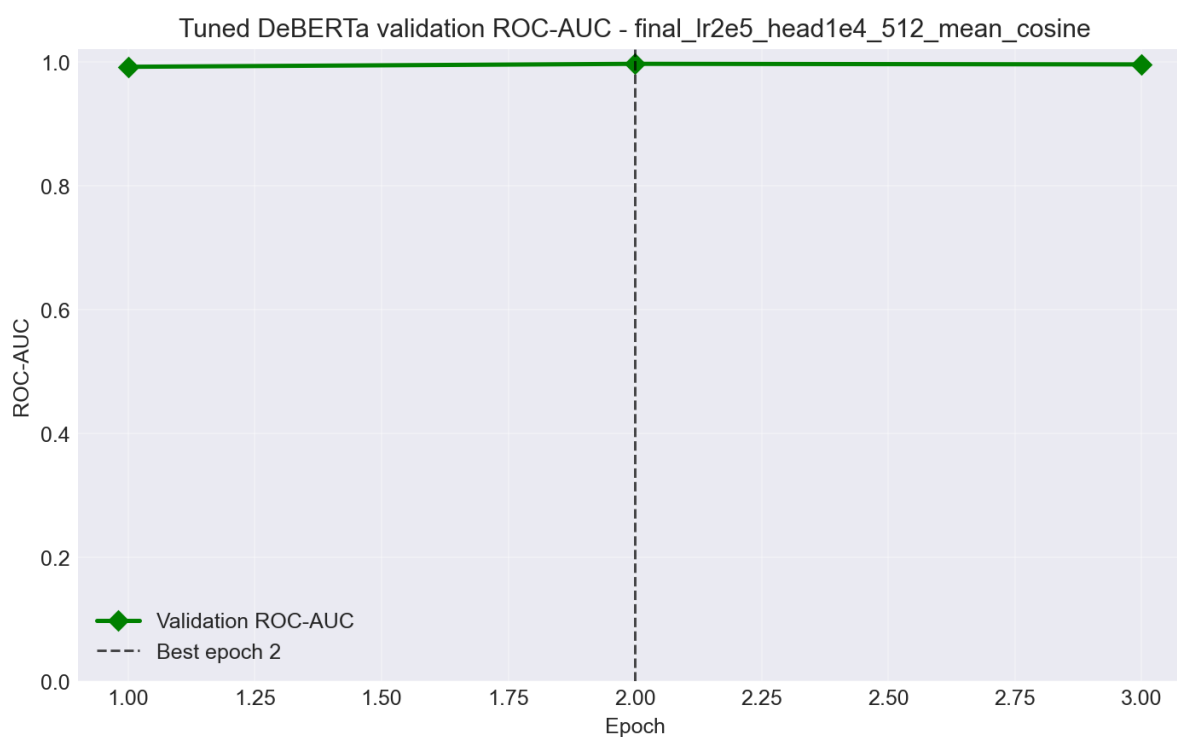
Hình 17: Precision-Recall curves so sánh các mô hình



Hình 18: Biểu đồ so sánh metrics tổng thể các mô hình



Hình 19: DeBERTa tuned — Training loss curve (nhìn từ ensemble notebook)



Hình 20: DeBERTa tuned — Validation AUC curve (nhìn từ ensemble notebook)

8.5. Nhận Xét Quan Trọng

1. **Baseline TF-IDF rất mạnh trên DAIGT-V2:** Logistic Regression và SGDClassifier đạt test ROC-AUC > 0.999 . Điều này cho thấy dataset DAIGT-V2 có các tín hiệu bề mặt (n-gram, pattern ký tự) khá rõ ràng để phân biệt Human/AI.
2. **DeBERTa tuned có ROC-AUC cao (0.998) nhưng precision thấp hơn** khi dùng threshold mặc định 0.5. Mô hình có xu hướng dự đoán AI mạnh, nên precision thấp hơn baseline TF-IDF.
3. **Simple Average đạt test ROC-AUC cao nhất (0.9997)** — cao hơn cả Weighted Ensemble (0.9995). Điều này cho thấy weighted ensemble có thể hơi **overfit theo validation fold**, còn simple average ổn định hơn trên test.
4. **Cần thận trọng khi diễn giải AUC ≈ 0.999 :** kết quả này là trên split hiện tại của DAIGT-V2. Không nên khẳng định mô hình tổng quát hoàn hảo cho mọi domain hoặc mọi LLM mới.

9. Weighted Ensemble

9.1. Công Thức

Weighted ensemble kết hợp xác suất của các mô hình:

$$P_{\text{final}}(\text{AI}) = w_1 \cdot P_{\text{LR}}(\text{AI}) + w_2 \cdot P_{\text{SGD}}(\text{AI}) + w_3 \cdot P_{\text{LightGBM}}(\text{AI}) + w_4 \cdot P_{\text{DeBERTa}}(\text{AI}) \quad (4)$$

Ràng buộc:

- $w_m \geq 0$ cho mọi m
- $\sum_{m=1}^M w_m = 1$

9.2. Tối Ưu Trọng Số

Trọng số được tối ưu bằng **Nelder-Mead** (`scipy.optimize.minimize`) để **maximize validation ROC-AUC**:

```

1 # Ham objective: minimize negative AUC
2 def objective(weights):
3     weights = softmax(weights) # Rang buoc tong = 1, w >= 0
4     preds = sum(w * p for w, p in zip(weights, model_probs))
5     return -roc_auc_score(labels, preds)
6
7 result = minimize(objective, x0=initial_weights, method='
    Nelder-Mead')
```

Tại sao dùng Nelder-Mead?

- Chỉ có 4 trọng số $\rightarrow$ không gian tìm kiếm nhỏ

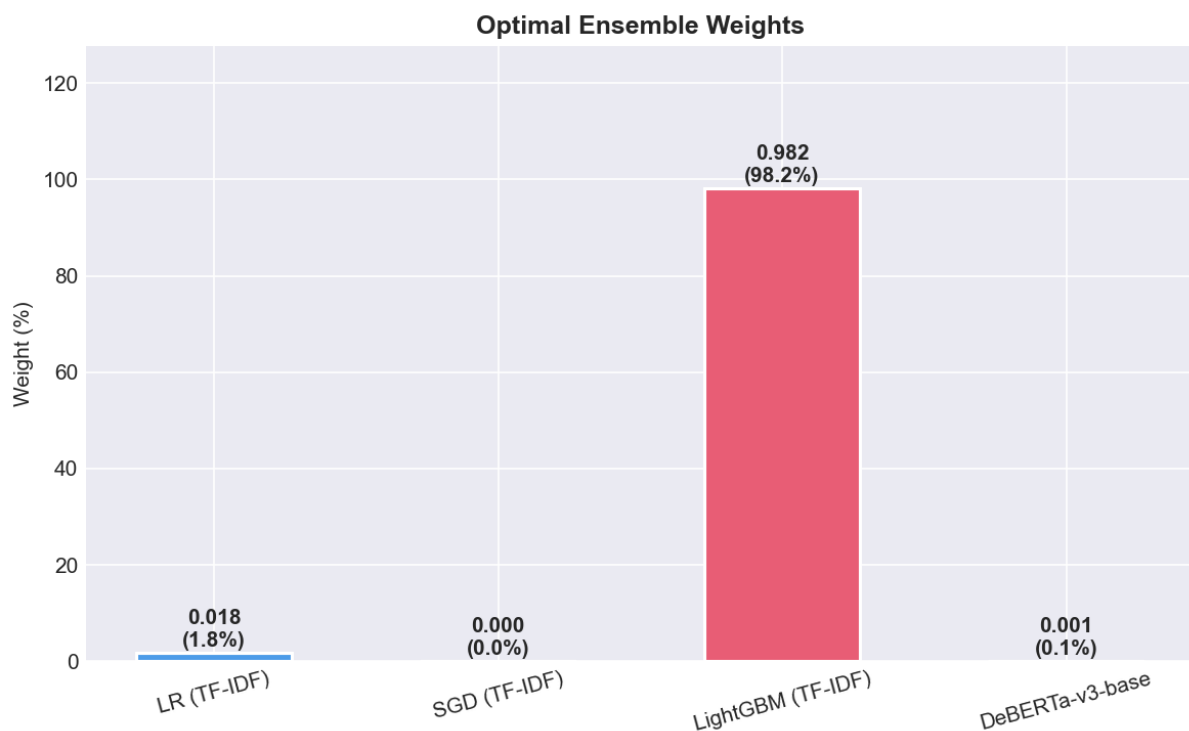
- AUC không differentiable → không dùng gradient-based method được
- Hội tụ nhanh, đơn giản

9.3. Trọng Số Hiện Tại

Trọng số trong outputs/ensemble/ensemble_config.json:

Model	Weight	ROC-AUC riêng
LR (TF-IDF)	0.0178	0.9996
SGD (TF-IDF)	0.0000	0.9996
LightGBM (TF-IDF)	0.9817	0.9996
DeBERTa-v3-base	0.0006	0.9972

Bảng 20: Trọng số ensemble tối ưu theo validation ROC-AUC



Hình 21: Biểu đồ trọng số ensemble của từng mô hình

9.4. Nhận Xét Trọng Số Ensemble

- Optimizer đặt trọng số lớn cho **LightGBM** vì trên validation fold, LightGBM kết hợp với một phần rất nhỏ LR và DeBERTa cho AUC cao nhất.
- **SGD bị đặt trọng số = 0** vì thông tin của nó đã bao hàm trong LR và LightGBM.
- **DeBERTa chỉ có trọng số 0.06%** — do AUC riêng thấp hơn baseline, optimizer giảm trọng số.

- Trên **test set**, Simple Average lại có AUC cao hơn Weighted Ensemble → weighted ensemble có thể hơi fit theo validation fold.

Bảng tổng kết AUC:

Cách kết hợp	Val AUC	Test AUC
Weighted Ensemble	0.9997	0.9995
Simple Average	0.9993	0.9997

Bảng 21: So sánh AUC giữa Weighted Ensemble và Simple Average

10. Error Analysis

10.1. Tổng Quan

Notebook task4_ensemble.ipynb sinh các file phân tích lỗi:

```
outputs/ensemble/ensemble_error_analysis.csv # Tất cả
predictions
outputs/ensemble/ensemble_top_errors.csv # Top 30 lỗi confidence
cao nhất
```

Trong 30 lỗi confidence cao nhất của Weighted Ensemble trên validation:

Loại lỗi	Số lượng	Ý nghĩa	Rủi ro
False Negative	25	Văn bản AI bị dự đoán là Human	Hệ thống bỏ sót nội dung AI-generated
False Positive	5	Văn bản Human bị dự đoán là AI	Rất nghiêm trọng — vụ oan bài viết thật

Bảng 22: Phân loại lỗi trong top 30 predictions có confidence cao nhất

10.2. Phân Tích False Positive (Human → AI)

Các mẫu False Positive có confidence từ 72% đến 95%:

#	True Label	Pred	P(AI)	Đoạn đầu văn bản
1	Human	AI	0.951	“When facing a difficult situation, you will of. . .”
2	Human	AI	0.888	“Life can get difficult sometimes, and we might. . .”
3	Human	AI	0.775	“Have you struggled and asked someone to help? . . .”

Bảng 23: Ví dụ các mẫu False Positive (Human bị dự đoán là AI)

Nguyên nhân gợi ý: Văn bản Human có **văn phong quá formal, cấu trúc câu quá đều, ít lỗi tự nhiên** — giống phong cách AI. Đây là trường hợp khó phân biệt nhất.

10.3. Phân Tích False Negative (AI $\rightarrow$ Human)

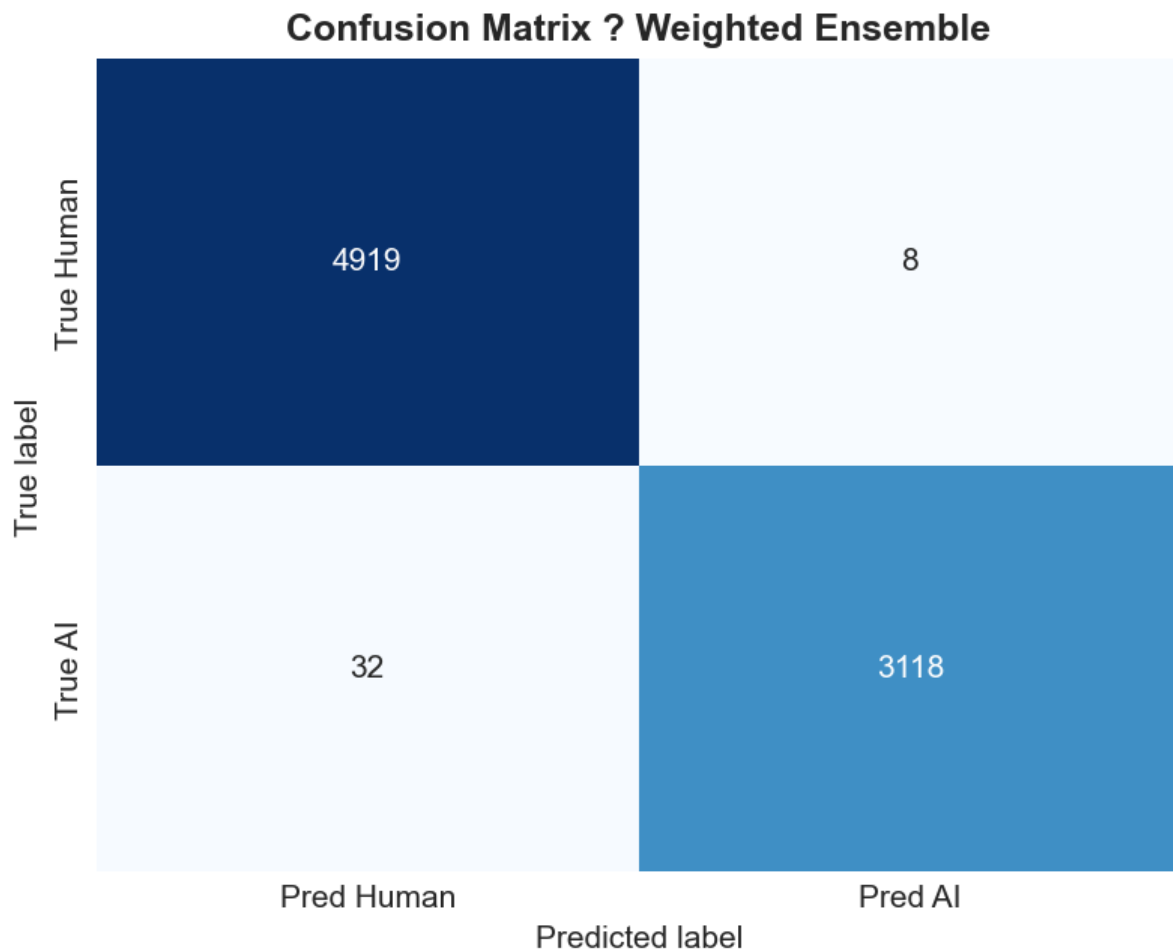
Các mẫu False Negative có confidence rất cao ($> 99\%$):

#	True Label	Pred	P(AI)	Đoạn đầu văn bản
1	AI	Human	0.0009	“Nobody ever said that writing an argumentative. . .”
2	AI	Human	0.0010	“My first opinion, would be that to be a goo. . .”
3	AI	Human	0.0010	“Dear: I believe that I can achieve your att. . .”

Bảng 24: Ví dụ các mẫu False Negative (AI bị dự đoán là Human)

Nguyên nhân gợi ý: Văn bản AI có **ví dụ cá nhân, phong cách tự nhiên**, hoặc **đoạn văn ngắn** — giống phong cách Human. Một số lỗi có thể đến từ việc văn bản bị truncate ở `max_length = 512`.

10.4. Confusion Matrix



Hình 22: Confusion matrix của Weighted Ensemble trên validation set

11. Web UI Demo

11.1. Tổng Quan

Ứng dụng web được xây dựng bằng **Gradio** tại `web_ui/app.py`.

Ứng dụng load checkpoint DeBERTa đã fine-tune:

- **Checkpoint:** `outputs/checkpoints/deberta_tuned_best.pt`
- **Config:** `outputs/tv2_deberta_tuning/best_config.json`

11.2. Luồng Xử Lý

Listing 3: Luồng xử lý inference của Web UI

```

Nguoi dung nhap van ban
      |
      v
Tokenize bang DeBERTa tokenizer
  
```

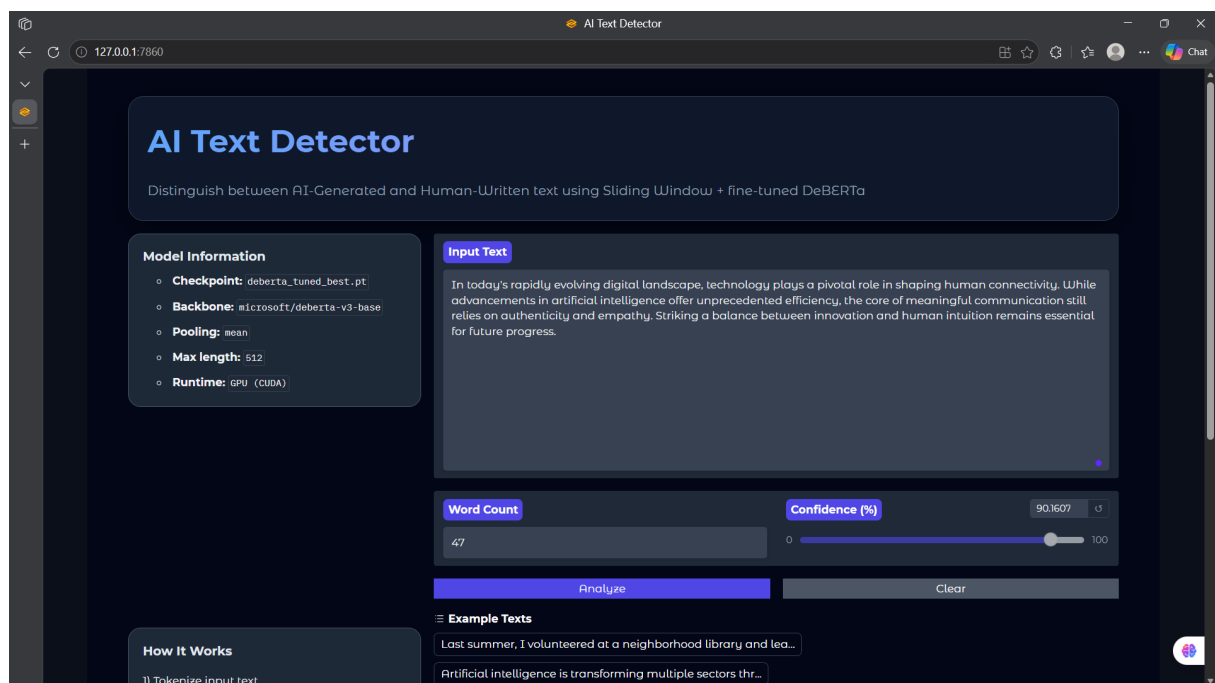
```

      |
      v
Model DeBERTa tuned best sinh logits
      |
      v
Softmax chuyen logits -> P(Human), P(AI)
      |
      v
Hien thi: nhan, confidence, xac suat chi tiet

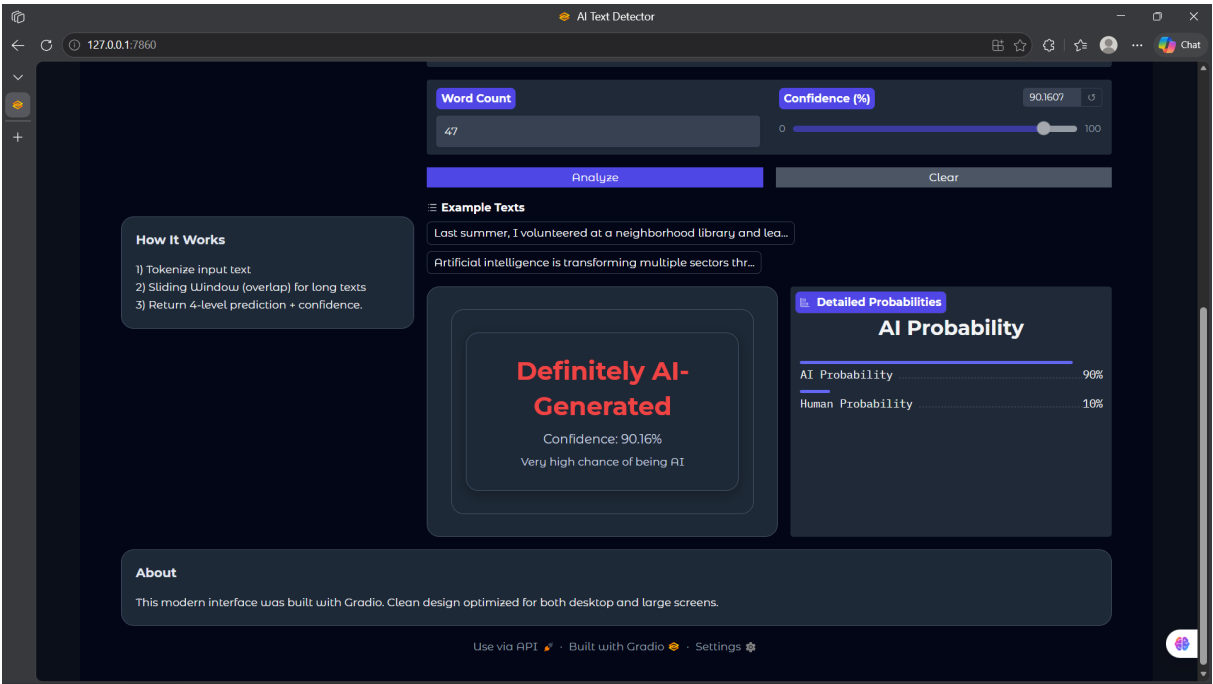
```

11.3. Tính Năng Giao Diện

- **Textbox:** Nhập văn bản dài tối đa 2,000 từ
- **Analyze button:** Chạy inference và hiển thị kết quả
- **Prediction card:** Nhấn màu (đỏ = AI, xanh = Human) + confidence %
- **Detailed probabilities:** Xác suất AI và Human chi tiết
- **Word count:** Đếm số từ trong văn bản
- **Clear button:** Xóa input và kết quả
- **Example texts:** 2 ví dụ mẫu để test nhanh
- **Dark mode UI:** Giao diện tối hiện đại
- **Auto GPU/CPU:** Tự dùng GPU (cuda) nếu có, fallback CPU



Hình 23: Demo Web UI



Hình 24: Demo Web UI

12. Chiến Lược Cải Thiện

12.1. High Impact

Chiến lược	Dự kiến cải thiện
5-Fold Cross Validation Huấn luyện 5 mô hình, sau đó ensemble OOF predictions	AUC tăng $\approx 0.005\text{--}0.01$
DeBERTa-v3-large Thay cho DeBERTa-v3-base nếu tài nguyên VRAM cho phép	AUC tăng $\approx 0.003\text{--}0.008$
Pseudo-labeling Dự đoán test set với confidence cao, thêm vào training data, sau đó retrain	AUC có thể tăng đáng kể nếu pseudo-label đủ chính xác
Data Augmentation Back-translation, sentence shuffle hoặc paraphrase nhẹ để tăng dữ liệu human	Cải thiện khả năng generalization

Bảng 25: Chiến lược cải thiện có tác động cao

12.2. Medium Impact

Chiến lược	Mô tả
Multi-model Ensemble: Thêm ELECTRA, RoBERTa-large vào ensemble	Đa dạng hóa model
Perplexity feature: Dùng GPT-2 tính perplexity làm feature cho LightGBM	Feature mạnh cho AI detection
Threshold tuning: Điều chỉnh threshold tối ưu theo F1 thay vì 0.5	Cải thiện precision
SWA: Stochastic Weight Averaging — average weights các checkpoint cuối	Ổn định hơn

Bảng 26: Chiến lược cải thiện có tác động trung bình

12.3. Experimental

Chiến lược	Mô tả
Adversarial Training (FGSM/PGD)	Cải thiện robustness
Calibration: Platt scaling	Cải thiện probability calibration
Gradient Checkpointing	Cho phép max_length > 512 khi VRAM hạn chế
Dynamic Padding	padding='longest' thay vì max_length để tăng tốc

Bảng 27: Chiến lược thử nghiệm (experimental)

13. Kết Luận

13.1. Hoàn Thành Các Yêu Cầu

Dự án đã hoàn thành các yêu cầu chính:

- ✓ Chọn dataset phù hợp: **DAIGT-V2**
- ✓ Thực hiện **EDA**, kiểm tra phân phối nhãn và đặc trưng văn bản
- ✓ Xây dựng **pipeline tiền xử lý**, tokenization và chia dữ liệu train/validation/test
- ✓ Huấn luyện **baseline truyền thống**: TF-IDF + Logistic Regression, SGDClassifier, LightGBM
- ✓ Fine-tune **DeBERTa-v3-base** với mean pooling, LLRD, cosine scheduler
- ✓ Lưu **checkpoint** tuned best và đánh giá trên held-out test set
- ✓ So sánh baseline, DeBERTa, simple average và **weighted ensemble**
- ✓ Sinh biểu đồ **ROC**, **PR**, **confusion matrix**, training loss/validation metrics
- ✓ Phân tích lỗi (**error analysis**) với false positive/negative

- ✓ Xây dựng **Web UI Gradio** sử dụng model DeBERTa đã fine-tune

13.2. Kết Quả Đáng Chú Ý

Mục	Kết quả
DeBERTa tuned test ROC-AUC	0.9982
DeBERTa tuned test F1	0.9159
DeBERTa tuned test Recall	0.9994
Best test ROC-AUC (bảng so sánh)	0.9997 — Simple Average
Best validation ROC-AUC (ensemble)	0.9997 — Weighted Ensemble
Thời gian huấn luyện DeBERTa (3 epochs, Kaggle GPU)	≈164.5 phút

Bảng 28: Tóm tắt kết quả nổi bật của dự án

13.3. Nhận Xét Tổng Thể

- **Baseline TF-IDF rất mạnh trên DAIGT-V2**, cho thấy dataset có các tín hiệu bề mặt rõ ràng. Điều này không bất thường với bài toán AI text detection, vì AI thường có pattern n-gram và ký tự đặc trưng.
- **DeBERTa** vẫn đóng vai trò mô hình Transformer chính để đáp ứng yêu cầu fine-tune LLM và phục vụ Web UI demo. Mô hình có ROC-AUC test rất cao (0.998).
- Kết quả nên được trình bày **khách quan**: mô hình đạt hiệu năng cao trên split hiện tại của DAIGT-V2, nhưng **cần đánh giá thêm trên dữ liệu ngoài domain** (ví dụ: văn bản từ GPT-4o, Claude 3.5, Llama 3) nếu muốn khẳng định khả năng tổng quát hóa trong môi trường thực tế.

A. Phụ Lục A: Bảng Thuật Ngữ

Thuật ngữ	Giải thích
ROC-AUC	Receiver Operating Characteristic — Area Under Curve. Đo khả năng phân biệt nhãn 0/1 với mọi ngưỡng quyết định. AUC = 1 là hoàn hảo, AUC = 0.5 là ngẫu nhiên.
PR-AUC	Precision-Recall Area Under Curve. Đánh giá chất lượng mô hình khi quan tâm đến cả precision và recall, phù hợp với dữ liệu lệch nhãn.
Tokenization	Quá trình phân tách văn bản thành các đơn vị con (subwords/tokens) để mô hình xử lý. DeBERTa dùng SentencePiece BPE tokenizer.
Attention Mask	Ma trận nhị phân [1 = token thật, 0 = padding] để mô hình không chú ý vào padding tokens.
Mean Pooling	Tính trung bình có trọng số của hidden states theo attention mask. Tốt hơn CLS pooling cho classification vì đại diện toàn bộ chuỗi.

Thuật ngữ	Giải thích
Type-Token Ratio (TTR)	Tỷ lệ số từ duy nhất / tổng số từ. AI thường có TTR thấp hơn human do xu hướng lặp pattern.
Disentangled Attention	Kỹ thuật tách content vector và position vector trong self-attention của DeBERTa, cải thiện khả năng học tương quan nội dung-vị trí.
LLRD	Layerwise Learning Rate Decay — gán LR thấp dần từ classifier head xuống các attention layer đầu để tránh catastrophic forgetting.
Gradient Accumulation	Gộp gradient từ K mini-batches trước khi cập nhật, tạo effective batch size lớn hơn khi VRAM hạn chế.
AMP (Mixed Precision)	Dùng float16 cho forward/backward (nhanh hơn), float32 cho optimizer update (chính xác hơn), với GradScaler để tránh underflow.
Early Stopping	Dừng training khi metric validation không cải thiện sau N epochs (patience), để tránh overfitting.
Out-of-Fold (OOF)	Predictions trên validation set thu được từ K-Fold CV, đảm bảo mọi mẫu đều là dự đoán unseen khi tối ưu ensemble weights.
Label Smoothing	Làm mềm one-hot labels sang phân phối mịn hơn để giảm overconfidence của mô hình.
TF-IDF	Term Frequency–Inverse Document Frequency. Biến text thành vector dựa trên tần suất từ/cụm từ/ký tự; từ phổ biến quá bị giảm điểm.
Nelder-Mead	Phương pháp tối ưu không cần gradient (simplex method), phù hợp với hàm objective không differentiable như AUC.
CrossEntropyLoss	Loss function cho classification; với 2 class, model trả logits [B, 2] và label là 0 hoặc 1.

Bảng 29: Bảng thuật ngữ kỹ thuật sử dụng trong báo cáo

B. Phụ Lục B: Danh Sách Hình Ảnh

#	Nội dung	File
1	Phân phối nhãn	outputs/eda/fig1_label_distribution.png
2	Phân phối số từ	outputs/eda/fig2_word_count_distribution.png
3	Phân phối features	outputs/eda/fig3_feature_distribution.png
4	Histogram word count	outputs/eda/fig4_word_count_histogram.png
5	Correlation heatmap	outputs/eda/fig5_correlation_heatmap.png
6	Scatter plot WC vs LD	outputs/eda/fig6_scatter_wc_vs_ld.png
7	Metadata distribution	outputs/eda/fig7_meta_distribution.png
8	Length category	outputs/eda/fig8_length_category.png
9	Token length distribution	outputs/pipeline/token_length_distribution.png
10	Training loss curve	outputs/tv2_deberta_tuning/.../best_run_loss_curve.png
11	Validation metrics	outputs/tv2_deberta_tuning/.../best_run_validation_metrics.png
12	Tuning loss curves (all configs)	outputs/tv2_deberta_tuning/tuning_loss_curves.png
13	Tuning metric curves	outputs/tv2_deberta_tuning/tuning_metric_curves.png
14	ROC curves	outputs/ensemble/roc_curves.png
15	PR curves	outputs/ensemble/pr_curves.png
16	Confusion matrix	outputs/ensemble/confusion_matrix_weighted_ensemble.png
17	Ensemble weights	outputs/ensemble/ensemble_weights.png
18	Model comparison	outputs/ensemble/ensemble_comparison.png
19	DeBERTa tuned loss (ens.	outputs/ensemble/deberta_training_process/deberta_tuned_loss_curve.png

C. Tài Liệu Tham Khảo

Các tài liệu dưới đây được chọn theo hai nguyên tắc: ưu tiên paper gốc hoặc tài liệu chính thức, và ưu tiên trang tài liệu còn được duy trì/cập nhật. Ngày truy cập: 29/05/2026.

C.1. Dataset Và Bài Toán AI-generated Text Detection

- [1] Kaggle. *LLM - Detect AI Generated Text*. Kaggle Competition. 2023. URL: <https://www.kaggle.com/competitions/llm-detect-ai-generated-text>
Vai trò trong đề án: tham chiếu bài toán phát hiện văn bản AI-generated, dạng input/output và bối cảnh đánh giá bằng xác suất.
- [2] TheDrCat. *DAIGT V2 Train Dataset*. Kaggle Dataset. URL: <https://www.kaggle.com/datasets/thedrcat/daigt-v2-train-dataset>
Vai trò trong đề án: nguồn dữ liệu chính sử dụng trong project, gồm văn bản Human và AI-generated.
- [3] Tang, R., Chuang, Y.-N., Hu, X. *The Science of Detecting LLM-Generated Texts*. arXiv:2303.07205. 2023. URL: <https://arxiv.org/abs/2303.07205>
Vai trò trong đề án: tài liệu nền về các hướng tiếp cận phát hiện văn bản do LLM sinh ra, các thách thức về generalization và robustness.
- [4] Crothers, E., Japkowicz, N., Viktor, H. L. *A Survey of AI-generated Text Forensic Systems: Detection, Attribution, and Characterization*. arXiv:2403.01152. 2024. URL: <https://arxiv.org/abs/2403.01152>
Vai trò trong đề án: survey mới hơn về hệ thống forensic cho văn bản AI-generated, dùng để bổ sung phần hạn chế và hướng phát triển.
- [5] SCITEPRESS. *A Comparative Study of ML Approaches for Detecting AI-Generated Text*. 2025. URL: <https://www.scitepress.org/Papers/2025/135702/135702.pdf>
Vai trò trong đề án: tài liệu tham khảo cập nhật về so sánh các phương pháp machine learning cho AI-generated text detection.

C.2. Transformer, DeBERTa Và Fine-tuning

- [6] He, P., Liu, X., Gao, J., Chen, W. *DeBERTa: Decoding-enhanced BERT with Disentangled Attention*. arXiv:2006.03654. 2020. URL: <https://arxiv.org/abs/2006.03654>
Vai trò trong đề án: paper gốc mô tả disentangled attention và enhanced mask decoder của DeBERTa.
- [7] He, P., Gao, J., Chen, W. *DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing*. arXiv:2111.09543 / ICLR 2023. URL: <https://arxiv.org/abs/2111.09543>
Vai trò trong đề án: paper nền cho DeBERTa-v3, giải thích replaced token detection và gradient-disentangled embedding sharing.
- [8] Microsoft. *DeBERTa official implementation*. GitHub Repository. URL: <https://github.com/microsoft/DeBERTa>
Vai trò trong đề án: nguồn chính thức của Microsoft cho DeBERTa/DeBERTa-v3 và các model pretrained.

- [9] Hugging Face. *microsoft/deberta-v3-base model card*. Hugging Face Model Hub. **URL:** <https://huggingface.co/microsoft/deberta-v3-base>
Vai trò trong đề án: thông tin model pretrained dùng trong project, cách load tokenizer/model bằng Transformers.
- [10] Hugging Face. *Transformers: Text classification guide*. Official Documentation. **URL:** https://huggingface.co/docs/transformers/main/tasks/sequence_classification
Vai trò trong đề án: tham chiếu quy trình fine-tune Transformer cho bài toán sequence/text classification.

C.3. PyTorch Training Pipeline

- [11] PyTorch. *torch.utils.data: Dataset and DataLoader*. Official Documentation. **URL:** <https://docs.pytorch.org/docs/stable/data.html>
Vai trò trong đề án: tham chiếu cho việc xây dựng Dataset, DataLoader, batching, shuffle và data loading.
- [12] PyTorch. *Automatic Mixed Precision examples*. Official Documentation. Cập nhật 2024. **URL:** https://docs.pytorch.org/docs/stable/notes/amp_examples.html
Vai trò trong đề án: tham chiếu cho AMP, autocast, GradScaler và gradient accumulation.
- [13] PyTorch. *AdamW optimizer*. Official Documentation. **URL:** <https://docs.pytorch.org/docs/stable/generated/torch.optim.adamw.AdamW.html>
Vai trò trong đề án: tham chiếu cho AdamW, weight decay và các tham số optimizer trong fine-tuning.

C.4. Baseline TF-IDF, Logistic Regression, SGD Và LightGBM

- [14] scikit-learn. *TfidfVectorizer*. Official Documentation. **URL:** https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html
Vai trò trong đề án: tham chiếu cho word/character n-gram TF-IDF features trong baseline.
- [15] scikit-learn. *LogisticRegression*. Official Documentation. **URL:** https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
Vai trò trong đề án: tham chiếu cho baseline TF-IDF + Logistic Regression.
- [16] scikit-learn. *SGDClassifier*. Official Documentation. **URL:** https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.SGDClassifier.html
Vai trò trong đề án: tham chiếu cho baseline TF-IDF + SGDClassifier.
- [17] LightGBM. *LGBMClassifier*. Official Documentation. **URL:** <https://lightgbm.readthedocs.io/en/latest/pythonapi/lightgbm.LGBMClassifier.html>
Vai trò trong đề án: tham chiếu cho baseline TF-IDF + LightGBM và các tham số boosting.

C.5. Metrics Và Đánh Giá Mô Hình

- [18] scikit-learn. *roc_auc_score*. Official Documentation. URL: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_auc_score.html
Vai trò trong đề án: tham chiếu cho ROC-AUC, metric chính dùng để chọn checkpoint và so sánh mô hình.
- [19] scikit-learn. *sklearn.metrics API Reference*. Official Documentation. URL: <https://scikit-learn.org/stable/api/sklearn.metrics.html>
Vai trò trong đề án: tham chiếu cho accuracy, precision, recall, F1, ROC curve, PR curve và confusion matrix.

C.6. Web UI Và Triển Khai Demo

- [20] Gradio. *Blocks documentation*. Official Documentation. URL: <https://www.gradio.app/docs/gradio/blocks>
Vai trò trong đề án: tham chiếu cho việc xây dựng giao diện web linh hoạt bằng Gradio Blocks.
- [21] Gradio. *Interface documentation*. Official Documentation. URL: <https://www.gradio.app/main/docs/gradio/interface>
Vai trò trong đề án: tham chiếu cách đóng gói hàm dự đoán thành web demo đơn giản.

C.7. Ghi Chú Về Tính Cập Nhật

- Các paper DeBERTa/DeBERTa-v3 không phải tài liệu mới nhất theo năm xuất bản, nhưng là tài liệu gốc bắt buộc vì mô hình trong đề án dùng trực tiếp `microsoft/deberta-v3-base`.
- Các tài liệu thư viện được ưu tiên là official documentation bản stable/main hiện tại: Hugging Face Transformers, PyTorch, scikit-learn, LightGBM và Gradio.
- Các tài liệu survey năm 2023-2025 được dùng để cập nhật bối cảnh AI-generated text detection, hạn chế của detector và vấn đề generalization.